



This is the Thesis Title

*A thesis submitted in partial fulfillment of the requirements of the B.Sc.
degree in Electrical and Electronic Engineering*

Submitted by:

Student Name (Index)

Student Name (Index)

Supervised by:

Supervisor name

To

Department of Electrical and Electronic Engineering

Faculty of Engineering

University of Khartoum

September 2024

Declaration of Authorship

You can check the official EEE department thesis guidelines from the main menu "**thesis Guidelines .pdf**"

Dedication

Acknowledgments

Abstract

Retrieval-Augmented Generation (RAG) systems depend on effective information retrieval to ground Large Language Model outputs in external knowledge, yet Arabic information retrieval faces severe challenges stemming from morphological richness, orthographic variation, and vocabulary mismatch between queries and documents. Query enhancement—the modification of a user’s query before retrieval—offers a modular, retriever-agnostic intervention to address these challenges. However, existing query enhancement techniques were developed for English using proprietary models with 175 billion or more parameters, and their transferability to Arabic using small, open-source models has not been systematically investigated.

This thesis investigates the extent to which small open-source LLMs can improve Arabic information retrieval through query enhancement, and identifies the model characteristics that determine effectiveness. Experiments were conducted on the MIRACL Arabic benchmark, comprising 2,896 queries and 2.06 million passages, using mDPR for dense retrieval and BM25S for sparse retrieval. Error analysis on the baseline identified a 39% query failure rate and a significant short-query performance gap, motivating the adaptation of the Query2Doc technique for Arabic zero-shot application. Ten open-source LLMs with 2–8 billion parameters were evaluated using a standardised protocol, with all experiments executed on Google Colab GPUs.

Query enhancement improved dense retrieval for all nine viable models, with NDCG@10 improvements ranging from +3.7% to +23.5%. Aya Expanse 8B achieved the highest overall improvement (+23.5% dense, +9.2% BM25), while Jais-2 8B was the most effective model for sparse retrieval (+10.8%). Zero-shot Arabic query enhancement with a 3-billion-parameter model exceeded the original Query2Doc paper’s few-shot English results obtained with a 175-billion-parameter model. However, BM25 retrieval was degraded by six of nine

models due to term dilution, demonstrating that query enhancement strategies must be adapted to the retrieval paradigm. Model size was positively correlated with dense retrieval improvement, and training data diversity was found to be a more significant predictor of query expansion quality than Arabic-specific benchmark scores.

The proposed corpus-steered pipeline, combined with hybrid BM25+Dense fusion, achieved 0.7137 NDCG@10 on the MIRACL Arabic benchmark—a 54.5% improvement over the BM25 baseline and a 13.9% improvement over a strong no-QE hybrid system.

المستخلص

تعتمد أنظمة التوليد المعزّز بالاسترجاع على فعالية استرجاع المعلومات لربط مخرجات النماذج اللغوية الكبيرة بالمعرفة الخارجية، غير أن استرجاع المعلومات باللغة العربية يواجه تحديات جوهرية ناجمة عن الثراء الصرفي والتباين الإملائي وعدم التطابق المعجمي بين الاستعلامات والمستندات. يُمثّل تحسين الاستعلام---أي تعديل استعلام المستخدم قبل عملية الاسترجاع---تدخلاً نمطياً مستقلاً عن نوع المسترجع لمعالجة هذه التحديات. إلا أن تقنيات تحسين الاستعلام الحالية طُوّرت للغة الإنجليزية باستخدام نماذج مملوكة تتجاوز 175 مليار معامِل، ولم تُدرَس بشكل منهجي إمكانية نقلها إلى اللغة العربية باستخدام نماذج صغيرة مفتوحة المصدر.

تبحث هذه الأطروحة في مدى قدرة النماذج اللغوية الكبيرة الصغيرة مفتوحة المصدر على تحسين استرجاع المعلومات العربية من خلال تحسين الاستعلام، وتحديد خصائص النموذج التي تؤثر في الفعالية. أُجريت التجارب على معيار MIRACL العربي المكوّن من 2,896 استعلاماً و06.2 مليون مقطع نصي، باستخدام mdPR للاسترجاع الكثيف وBM25S للاسترجاع المتناثر. كشف تحليل الأخطاء على خط الأساس عن معدل فشل بلغ 39% وفجوة أداء ذات دلالة إحصائية للاستعلامات القصيرة، مما حفّز تكييف تقنية Query2Doc للتطبيق العربي بدون أمثلة تدريبية. قُيِّمت عشرة نماذج لغوية كبيرة مفتوحة المصدر بأحجام تتراوح بين 2 و8 مليارات معامِل باستخدام بروتوكول تقييم موحد، مع تنفيذ جميع التجارب على وحدات معالجة الرسومات في Colab. Google

حقّق تحسين الاستعلام تحسناً في الاسترجاع الكثيف لجميع النماذج التسعة القابلة للاستخدام، بنسب تحسّن في NDCG@10 تراوحت بين 7%.3+ و5%.23+. حقّق نموذج Expanse Aya 8B أعلى تحسّن إجمالي (+5%.23 كثيف، +2%.9 BM25)، بينما كان 8B Jais-2 الأكثر فعالية للاسترجاع المتناثر (+8%.10). تجاوز تحسين الاستعلام العربي بدون أمثلة تدريبية باستخدام نموذج بحجم 3 مليارات معامِل النتائج الأصلية لورقة Query2Doc التي استخدمت نموذجاً بحجم 175 مليار معامِل مع أمثلة تدريبية. غير أن أداء BM25 تراجع مع ستة من تسعة نماذج بسبب تخفيف المصطلحات، مما يؤكد ضرورة تكييف استراتيجيات تحسين الاستعلام وفقاً لنموذج الاسترجاع المستخدم. ارتبط حجم النموذج إيجابياً بتحسّن الاسترجاع الكثيف، وتبيّن أن تنوّع بيانات التدريب مؤثر أهم لجودة توسيع الاستعلام من درجات المعايير العربية المتخصصة.

حققت منظومة التوسيع الموجّه بالمدوّنة المقترحة، مقترنةً بالدمج الهجين بين BM25 والاسترجاع الكثيف، قيمة 7137.0 في $NDCG@10$ على معيار MIRACL العربي --بتحسّن نسبته 54.5%. مقارنةً بخط أساس BM25 وتحسّن نسبته 13.9% مقارنةً بنظام هجين قوي بدون تحسين للاستعلام.

List of Tables

2.1	Summary of language models used in the experiments.	23
3.1	LLM generation parameters for the initial Query2Doc experiment (Experiment 003).	38
3.2	Model configurations for the Query2Doc comparison experiments. All models used <code>max_new_tokens</code> = 128 and <code>top_p</code> = 0.9 unless otherwise noted.	41
4.1	Baseline retrieval results on MIRACL Arabic dev set (2,896 queries). Bold values indicate the higher score for each metric.	51
4.2	Performance segmentation of baseline queries by NDCG@10 score.	53
4.3	Retrieval performance by query length bucket.	53
4.4	Proportion of queries with at least one relevant passage at each retrieval depth.	53
4.5	Query2Doc results with dense retrieval (mDPR). Improvement is computed relative to the dense baseline (Experiment 001).	55
4.6	Query2Doc results with BM25 retrieval (BM25S). Negative values indicate performance degradation.	55
4.7	Comparison between the original Query2Doc results (English, MS-MARCO) and this implementation (Arabic, MIRACL).	56
4.8	Dense retrieval (mDPR) results for all models, ranked by NDCG@10. The improvement column is relative to the no-enhancement baseline (Experiment 001).	57
4.9	BM25 retrieval (BM25S) results for all models, ranked by NDCG@10. The improvement column is relative to the no-enhancement baseline (Experiment 002). Negative values indicate degradation.	58
4.10	Generational comparison within the Qwen model family.	61
4.11	Number of models improving over baseline, by retriever type.	62

4.12	Summary of all experiments. Exp. 001 and 002 are baselines; Exp. 003–009 use Query2Doc; Exp. 011–021 are Phase 4 expanded experiments. Bold values indicate the best result for each metric across all experiments. . .	64
4.13	BM25 nDCG@10 for nine models under query repetition. Columns $n \in \{1, 3, 5, 7, 10\}$ are fixed-count repetition (Query2Doc style); columns $\beta \in \{2, 4, 6\}$ are MuGI adaptive repetition where n is computed per-query as $n = \max(1, \lfloor d /(q \cdot \beta) \rfloor)$. MIRACL Arabic dev (2,896 queries). Best value per model in bold.	65
4.14	BM25 retrieval metrics at each model’s best repetition configuration. Δ nDCG is the improvement from $n = 1$ (no repetition) to the best configuration.	65
4.15	Hybrid retrieval fusion results on the MIRACL Arabic dev set (2,896 queries). CC = Convex Combination (min-max normalised per query, α controls Dense weight); RRF = Reciprocal Rank Fusion.	67
4.16	nDCG@10, Recall@10, Recall@100, and MRR for CSQE-enhanced retrieval on the MIRACL Arabic dev set. CSQE config: $k = 5$ first-pass, 2 corpus + 2 blind samples, $\alpha = 4$, Aya Expanse 8B.	69
4.17	BM25 retrieval metrics for corpus-only, blind-only, and combined CSQE expansion variants. All variants use $\alpha = 4$, Aya Expanse 8B, $k = 5$ first-pass.	69
4.18	Effect of query repetition factor α on CSQE BM25 retrieval nDCG@10. .	70
4.19	Results for three CSQE application strategies in hybrid fusion. Config A: BM25 receives CSQE query, Dense receives original query. Config B: BM25 receives original query, Dense receives CSQE query. Config C: both retrievers receive CSQE query.	71
4.20	Config A RRF ablation: contribution of corpus and blind expansion components when fused with the original Dense run.	71
4.21	Effect of query repetition factor α on the best Config A RRF system. All values use the same CSQE 2c+2b expansion, varying only the α prepended to the BM25-side query.	72
4.22	nDCG@10 improvements of Config A RRF over prior systems.	72
4.23	nDCG@10 progression from baseline to best system on the MIRACL Arabic dev set (2,896 queries). Δ cumulative measured from BM25S baseline.	73

4.24	Per-query comparison of the CSQE+Hybrid system (Config C RRF, 0.6936 nDCG@10) against the Aya Blind BM25 $n = 1$ baseline (0.5046 nDCG@10) on 2,896 MIRACL Arabic dev queries.	74
4.25	System performance split by whether BM25 first-pass ($k = 5$) retrieved a relevant document.	74
4.26	System performance decomposed by the number of tokens in the original MIRACL query.	75
4.27	Representative big-win queries illustrating the corpus grounding effect. In all three cases, CSQE achieved nDCG@10 = 1.000 while the blind baseline achieved 0.000.	75
4.28	Classification of 367 regression queries (CSQE nDCG@10 < Blind by > 0.1).	76

List of Figures

3.1	Overview of the experimental pipeline. The query enhancement module is the variable component across experiments; the retrieval and evaluation components remain fixed.	32
4.1	Dense retrieval NDCG@10 for all models. The dashed line indicates the no-enhancement baseline (0.4993). ALLaM is shown separately as a dropped model.	58
4.2	Relationship between model parameter count and dense retrieval NDCG@10 improvement. ALLaM-7B is excluded as an outlier due to tokeniser artefacts.	60
4.3	Heatmap of BM25 nDCG@10 across nine models and eight repetition configurations. Darker shading indicates higher performance. The optimal region shifts rightward (higher repetition) for smaller models.	66
4.4	Hybrid fusion nDCG@10 as a function of CC α . The horizontal dashed lines indicate RRF $k = 20$ (0.6267) and RRF $k = 60$ (0.6230). The dotted lines show the BM25 (0.4621) and mDPR (0.4993) baselines.	68
4.5	Per-query nDCG@10 scatter plot comparing CSQE+Hybrid (Config C RRF) against the Blind BM25 baseline. Points above the diagonal indicate queries where CSQE improved retrieval; points below indicate regressions.	74
4.6	Distribution of regression root causes across 367 queries where CSQE+Hybrid underperformed the blind baseline by more than 0.1 nDCG@10.	76

Contents

List of Tables	viii
List of Figures	xi
Contents	xii
List of Abbreviations	xvi
1 Introduction	1
1.1 Problem Definition	2
1.2 Objectives	3
1.3 Thesis Layout	4
2 Theoretical Background and Literature Review	6
2.1 Theoretical Background	6
2.1.1 Large Language Models and the Transformer Architecture	6
2.1.2 Retrieval-Augmented Generation	8
2.1.3 Information Retrieval Methods	9
2.1.4 Query Enhancement Techniques	11
2.1.5 Arabic Language Processing Challenges	12
2.2 Mathematical Models	13
2.2.1 BM25 Scoring Function	13
2.2.2 Dense Retrieval and Cosine Similarity	14
2.2.3 Evaluation Metrics	14
2.3 Models Used in Experiments	16
2.3.1 Arabic-Specialized Models	16
2.3.2 Multilingual Models	19
2.3.3 Experimental Models	21

2.3.4	Retrieval Models	22
2.3.5	Model Comparison Summary	23
2.4	Related Work	23
2.4.1	Foundational Query Enhancement Research (2022–2023)	24
2.4.2	Modern LLM-Based Query Enhancement (2024–2025)	25
2.4.3	Arabic Information Retrieval and RAG	26
2.4.4	Research Gap	27
2.5	Chapter Summary	29
3	Methodology	30
3.1	Dataset and Experimental Setup	30
3.1.1	Dataset: MIRACL Arabic	30
3.1.2	Hardware and Software Environment	31
3.1.3	Evaluation Metrics	32
3.1.4	Experimental Pipeline Overview	32
3.2	Baseline Implementation	33
3.2.1	Dense Retrieval Baseline (mDPR)	33
3.2.2	Sparse Retrieval Baseline (BM25S)	34
3.2.3	Baseline Comparison Rationale	34
3.3	Error Analysis Methodology	35
3.3.1	Quantitative Analysis Framework	35
3.3.2	Query Length Bucketing	36
3.3.3	Failed Query Inspection	36
3.4	Query2Doc Implementation	36
3.4.1	Query2Doc Technique	37
3.4.2	Modifications from the Original Paper	37
3.4.3	LLM Configuration	38
3.4.4	Batch Processing Optimisation	38
3.4.5	Dense Retrieval with Query2Doc	39
3.4.6	BM25 Retrieval with Query2Doc	39
3.5	Model Comparison Methodology	40
3.5.1	Model Selection Criteria	40
3.5.2	Standardised Evaluation Protocol	41

3.5.3	Temperature Selection	42
3.5.4	Model-Specific Technical Issues	42
3.5.5	Quantisation Strategy	43
3.6	Query Repetition for Sparse Retrieval	44
3.7	Hybrid Retrieval Fusion	45
3.8	Corpus-Steered Query Expansion (CSQE)	46
3.8.1	The CSQE Pipeline	46
3.8.2	Component Ablation Design	47
3.8.3	Retriever-Specific Application	48
3.9	Per-Query Error Analysis	49
4	Results and Discussion	51
4.1	Baseline Results and Comparison	51
4.1.1	Complementary Strengths	52
4.2	Error Analysis Findings	52
4.2.1	Overall Failure Rate	52
4.2.2	Short Query Performance Gap	53
4.2.3	Retrieval Coverage Analysis	53
4.2.4	Technique Selection Rationale	54
4.3	Query2Doc Results	54
4.3.1	Dense Retrieval (Experiment 003)	54
4.3.2	BM25 Retrieval (Experiment 004)	55
4.3.3	Comparison with the Original Query2Doc Paper	56
4.4	Model Comparison Results	57
4.4.1	Dense Retrieval Leaderboard	57
4.4.2	BM25 Retrieval Results	58
4.4.3	Dropped Models Analysis	59
4.5	Key Findings and Analysis	60
4.5.1	Model Size Correlates with Dense Retrieval Performance	60
4.5.2	Generational Improvement Within Model Families	61
4.5.3	Arabic Specialisation vs. Multilingual Training	61
4.5.4	Dense vs. BM25 Behaviour with Query Enhancement	62
4.5.5	Best Model Recommendations	63

4.5.6	Summary of All Experiments	63
4.6	BM25 Query Repetition Results	64
4.7	Hybrid Retrieval Fusion Results	67
4.8	Corpus-Steered Query Expansion Results	68
4.8.1	Main CSQE Results	68
4.8.2	Component Ablation	69
4.9	CSQE with Hybrid Fusion	70
4.9.1	Fusion Configuration Comparison	70
4.9.2	Overall System Progression	72
4.10	Per-Query Error Analysis	73
4.10.1	Win/Loss Distribution	73
4.10.2	First-Pass Quality as Dominant Predictor	74
4.10.3	Big Wins: The Corpus Grounding Effect	75
4.10.4	Regression Analysis	76
5	Conclusion and Recommendations	78
5.1	Conclusions	78
5.2	Challenges	81
5.3	Recommendations for Future Work	82
	Bibliography	86

List of Abbreviations

TE **T**est **E**xample

AE **A**nother **E**xample

Chapter 1

Introduction

The rapid growth of digital Arabic content has intensified the need for intelligent information

retrieval systems capable of accurately locating relevant information within large corpora. Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge, mitigating hallucination by retrieving relevant passages before generation. However, the quality of a RAG system’s final output is fundamentally bounded by the quality of its retrieval component: if the retriever fails to surface relevant documents, the generator cannot compensate for this deficiency regardless of its capabilities.

Arabic information retrieval faces challenges that compound this retrieval bottleneck. The Arabic language employs a root-and-pattern morphological system in which a single root can yield dozens of surface forms, creating severe vocabulary mismatch between queries and documents. Orthographic variations in characters such as Hamza and Alif introduce further inconsistencies, while the diglossic nature of Arabic—where Modern Standard Arabic (MSA) is used in formal text but regional dialects dominate informal communication—creates a systematic gap between how users formulate queries and how knowledge bases are written. These linguistic characteristics amplify the retrieval failure rate beyond what is observed for English and other morphologically simpler languages.

Query enhancement—the modification of a user’s query before it is submitted to the retriever—offers a promising pre-retrieval intervention. Rather than modifying the retriever or reindexing the knowledge base, query enhancement operates as a modular layer that enriches the query with additional vocabulary, context, and semantic information. This approach is retriever-agnostic, requires no changes to the existing infrastructure, and can be deployed incrementally. Recent work has demonstrated that LLMs can generate effective query expansions by producing pseudo-documents that bridge the vocabulary

gap between queries and relevant passages.

However, existing query enhancement research is overwhelmingly English-centric and relies on proprietary models with 175 billion or more parameters. The foundational techniques—Query2Doc, HyDE, and GRF—were all developed and validated using GPT-3 or comparable commercial APIs. Whether these techniques transfer effectively to Arabic, and whether small open-source models running on consumer-grade hardware can serve as viable alternatives, has not been systematically investigated.

1.1 Problem Definition

The problem addressed in this thesis lies at the intersection of three gaps in the current state of Arabic information retrieval.

The retrieval gap. Arabic information retrieval systems exhibit high failure rates even on standard benchmarks. Dense and sparse retrieval methods, while achieving reasonable aggregate performance, fail on a substantial proportion of individual queries. Short queries, which carry insufficient information for the retriever to identify relevant passages, are disproportionately affected. This information poverty in short queries is a fundamental limitation that cannot be addressed by improving the retriever alone.

The language gap. LLM-based query enhancement techniques were developed for English and validated on English benchmarks using English-language models. Arabic presents distinct challenges—morphological richness, orthographic variation, and vocabulary mismatch—that may cause these techniques to behave differently. The effectiveness of generative query enhancement for Arabic has not been empirically established.

The resource gap. The query enhancement literature relies almost exclusively on proprietary models with hundreds of billions of parameters, accessed through commercial APIs. This creates a practical barrier for deployment in resource-constrained settings. Whether small open-source models with 2–8 billion parameters, executable on freely available cloud GPUs, can generate useful Arabic query expansions remains an open

question.

These three gaps motivate the central research question of this thesis: *To what extent can small, open-source LLMs improve Arabic information retrieval through query enhancement, and what model characteristics determine effectiveness?*

1.2 Objectives

The objectives of this thesis are as follows:

1. To establish retrieval baselines for Arabic information retrieval using both dense and sparse methods, tested independently to isolate the sources of improvement from subsequent query enhancement experiments.
2. To conduct systematic error analysis on the baseline results to identify the primary failure patterns in Arabic retrieval and guide the selection of query enhancement techniques.
3. To adapt and implement the Query2Doc query enhancement technique for Arabic zero-shot application using small open-source LLMs, including engineering optimisations for practical deployment on consumer-grade GPUs.
4. To evaluate and compare ten open-source LLMs with 2–8 billion parameters for Arabic query expansion quality using a standardised evaluation protocol across both dense and sparse retrieval paradigms.
5. To analyse the interaction between LLM-based query enhancement and retrieval paradigm to identify model characteristics—such as parameter count, training data composition, and vocabulary design—that predict query expansion effectiveness for Arabic.
6. To investigate query repetition as a technique for resolving the BM25 term-dilution degradation observed when pseudo-documents are concatenated with short Arabic queries, and to identify the optimal repetition factor across multiple LLM models.

7. To adapt and evaluate Corpus-Steered Query Expansion (CSQE) for Arabic retrieval, grounding LLM-generated expansions in first-pass retrieved documents and assessing the contribution of corpus-grounded versus blind expansion components through ablation.
8. To combine corpus-steered query expansion with hybrid BM25–Dense retrieval and determine the optimal retriever-specific query representation strategy, examining whether applying query expansion asymmetrically to one retriever outperforms applying it to both.

1.3 Thesis Layout

The remainder of this thesis is organised as follows:

Chapter 2 establishes the theoretical foundations required to understand the research. It introduces the core concepts of Large Language Models, Retrieval-Augmented Generation, information retrieval methods (sparse, dense, and hybrid), and query enhancement techniques. The mathematical formulations underlying the retrieval and evaluation methods are presented, followed by detailed descriptions of all models used in the experimental work. The chapter concludes with a review of related literature and the identification of the research gap addressed by this thesis.

Chapter 3 presents the methodology employed throughout the research. It describes the dataset (MIRACL Arabic), hardware and software environment, and evaluation metrics. The baseline implementation for both dense (mDPR) and sparse (BM25S) retrieval is detailed, followed by the error analysis methodology that guided technique selection. The adaptation of Query2Doc for Arabic zero-shot application is described, including modifications from the original paper and engineering optimisations. The chapter then presents the query repetition methodology for resolving BM25 term-dilution, the hybrid BM25–Dense fusion methodology, and the Corpus-Steered Query Expansion (CSQE) pipeline including its component ablation design and retriever-specific application strategy. The chapter concludes with the per-query error analysis methodology.

Chapter 4 reports the results and discussion in zigzag correspondence with the methodology. Baseline performance is analysed, error analysis findings are presented, and Query2Doc results are reported for both dense and sparse retrieval. The comprehensive model comparison leaderboards are presented and analysed, including the examination of dropped models. The chapter then reports the Phase 4 expanded results: query repetition sweep across nine models, hybrid retrieval fusion configurations, CSQE component ablation and alpha sweep, CSQE combined with hybrid fusion across three retriever assignment configurations, the overall system progression table, and a per-query error analysis decomposing the sources of improvement and regression.

Chapter 5 presents the overall conclusions, discusses the challenges encountered during the research, and provides recommendations for future work in Arabic query enhancement for RAG systems.

Chapter 2

Theoretical Background and Literature Review

This chapter establishes the theoretical foundations and reviews the current state-of-the-art relevant to the research presented in this thesis. Section 2.1 introduces the core concepts of Large Language Models (LLMs), Retrieval-Augmented Generation (RAG) systems, information retrieval methods, query enhancement techniques, and the specific challenges posed by the Arabic language. Section 2.2 presents the mathematical formulations underlying the retrieval and evaluation methods employed. Section 2.3 provides detailed descriptions of all models utilized in the experimental work. Finally, Section 2.4 surveys the related literature and identifies the research gap addressed by this thesis.

2.1 Theoretical Background

2.1.1 Large Language Models and the Transformer Architecture

Large Language Models (LLMs) are deep neural networks trained on vast corpora of text data to model the statistical properties of natural language. The modern era of LLMs was initiated by the introduction of the Transformer architecture by Vaswani et al. [1], which replaced recurrent processing with a self-attention mechanism that computes relationships between all positions in a sequence simultaneously.

The Transformer architecture consists of two main components: an encoder that processes input sequences into contextual representations, and a decoder that generates output sequences autoregressively. The core innovation is the *scaled dot-product attention*

mechanism, which computes attention weights between query (Q), key (K), and value (V) matrices as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2.1)$$

where d_k is the dimensionality of the key vectors. Multiple attention heads are employed in parallel, allowing the model to attend to information from different representation subspaces.

Two dominant architectural paradigms have emerged from the original Transformer design. *Encoder-only* models such as Bidirectional Encoder Representations from Transformers (BERT) process entire sequences bidirectionally and are well-suited for classification and retrieval tasks. *Decoder-only* models, which underpin most modern LLMs, generate text autoregressively by predicting each token conditioned on all preceding tokens.

The development of LLMs typically involves two stages. During *pre-training*, the model is trained on trillions of tokens from diverse text sources using self-supervised objectives such as next-token prediction. This stage imbues the model with broad linguistic and world knowledge. During *instruction tuning* (also called Supervised Fine-Tuning, or SFT), the pre-trained model is further trained on curated prompt-response pairs to follow user instructions. Alignment techniques such as Direct Preference Optimization (DPO) and Reinforcement Learning from Human Feedback (RLHF) are subsequently applied to improve helpfulness and safety [2].

A fundamental limitation of LLMs is that their knowledge is *parametric*—encoded statically in model weights during training. This makes the knowledge prone to becoming outdated, incomplete for specialized domains, and susceptible to *hallucination*: the generation of fluent but factually incorrect text. The model cannot distinguish between information it has reliably learned and information it is confabulating, nor can its knowledge be updated without costly retraining [3].

2.1.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. [3] to address the limitations of purely parametric language models. RAG augments an LLM with access to an external knowledge base by incorporating a retrieval step before generation, thereby grounding the model's outputs in verifiable sources.

The standard RAG architecture consists of two primary components:

- **The Retriever:** Given a user query q , the retriever searches a knowledge base $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$ to identify a set of relevant passages $\{d_1, d_2, \dots, d_k\}$, ranked by relevance to q . Retrieval methods are discussed in detail in Section 2.1.3.
- **The Generator:** An LLM that produces an answer a conditioned on both the original query q and the retrieved passages. The retrieved context provides factual grounding, reducing the likelihood of hallucination and enabling the model to cite sources.

The RAG pipeline operates sequentially: (1) the user submits a query q ; (2) the retriever identifies the top- k relevant passages from the knowledge base; (3) the generator produces an answer conditioned on the query and retrieved passages; and (4) the system returns the generated answer, optionally with source citations. This architecture effectively separates knowledge storage (in the external corpus) from language generation (in the LLM), allowing the knowledge base to be updated independently without retraining the model.

The quality of the final generated answer is fundamentally bounded by the quality of the retrieved passages. If the retriever fails to surface relevant documents, the generator cannot compensate for this deficiency regardless of its capabilities. This *retrieval bottleneck* motivates the investigation of techniques to improve retrieval effectiveness, which is the central focus of this thesis.

2.1.3 Information Retrieval Methods

Information retrieval methods can be broadly categorized into sparse, dense, and hybrid approaches, each with distinct strengths and limitations.

2.1.3.1 Sparse Retrieval

Sparse retrieval methods represent queries and documents as high-dimensional sparse vectors, where each dimension corresponds to a term in the vocabulary and the value reflects the term's importance. The most widely used sparse retrieval method is Best Matching 25 (BM25), a probabilistic ranking function based on the bag-of-words model [4]. BM25 computes relevance scores using term frequency, inverse document frequency, and document length normalization. The mathematical formulation of BM25 is presented in Section 2.2.1.

Sparse methods excel at exact term matching and are computationally efficient, requiring no neural network inference. However, they suffer from the *vocabulary mismatch problem*: if the query and document use different terms to express the same concept, sparse methods fail to identify the relevance. This limitation is particularly acute for morphologically rich languages such as Arabic, where a single concept may be expressed through numerous surface forms.

2.1.3.2 Dense Retrieval

Dense retrieval methods address the vocabulary mismatch problem by encoding queries and documents into continuous vector representations (embeddings) within a shared semantic space. Karpukhin et al. [5] introduced Dense Passage Retrieval (DPR), which employs a bi-encoder architecture: two independent BERT-based encoders produce fixed-size dense vectors for queries and passages respectively. Relevance is computed via vector similarity (typically dot product or cosine similarity, as formulated in Section 2.2.2) between the query embedding and passage embeddings.

Dense retrievers are trained on relevance-annotated data using contrastive learning objectives, where the model learns to place relevant query-passage pairs closer together in the embedding space while pushing irrelevant pairs apart. At inference time, approximate nearest neighbor search algorithms (e.g., FAISS) enable efficient retrieval over millions of passages.

While dense retrieval captures semantic relationships that sparse methods miss, it can struggle with rare terms, entity names, and highly specific queries where exact lexical matching is important [6].

2.1.3.3 Hybrid Retrieval

Hybrid retrieval combines sparse and dense methods to leverage their complementary strengths. Sparse retrieval excels at exact matching and handles rare terms well, while dense retrieval captures semantic similarity and handles paraphrasing. Common fusion strategies include Reciprocal Rank Fusion (RRF), which combines ranked lists from both methods, and score-level interpolation, which computes a weighted sum of sparse and dense relevance scores.

Reciprocal Rank Fusion (RRF) [7] combines ranked lists by summing inverse rank scores:

$$s_{\text{RRF}}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)} \quad (2.2)$$

where $\mathcal{R}$ is the set of ranked lists being fused and k is a smoothing constant (typically $k = 20$) that reduces the impact of top-ranked documents. RRF is parameter-free in the sense that it does not require score normalisation, making it robust to score distribution differences between sparse and dense retrievers.

Score-level interpolation (Convex Combination, CC) computes a weighted sum of normalised retriever scores:

$$s_{\text{CC}}(d, q) = \alpha \cdot \hat{s}_{\text{Dense}}(d, q) + (1 - \alpha) \cdot \hat{s}_{\text{BM25}}(d, q) \quad (2.3)$$

where $\hat{s}$ denotes min-max normalised scores and $\alpha \in [0, 1]$ controls the relative contribution

of the dense retriever. CC is sensitive to the normalisation strategy and score scale differences between the two systems.

2.1.4 Query Enhancement Techniques

Query enhancement refers to a family of techniques that modify the user's query before it is submitted to the retriever, with the goal of improving retrieval effectiveness. Song and Zheng [2] categorize query optimization methods into four atomic operations: *expansion*, *decomposition*, *disambiguation*, and *abstraction*. This thesis focuses primarily on query expansion, which is described below.

Query Expansion broadens the scope of a query by adding related terms, context, or generated text. Traditional query expansion methods such as Relevance Model 3 (RM3) extract expansion terms from pseudo-relevant documents retrieved in a first pass. Modern approaches leverage LLMs to generate expansion content, which can take several forms:

- **Hypothetical Document Embeddings (HyDE):** Proposed by Gao et al. [8], HyDE instructs an LLM to generate a hypothetical document that would answer the query. This generated document is then encoded using an unsupervised contrastive encoder, and the resulting embedding is used for retrieval. The encoder acts as a lossy compressor that filters out hallucinated details while retaining relevant semantic features.
- **Query2Doc:** Introduced by Wang et al. [6], this method generates a pseudo-document using an LLM and concatenates it with the original query to form an enriched query representation. For sparse retrieval, the original query is repeated multiple times before concatenation to prevent term dilution by the longer pseudo-document.
- **Generative Relevance Feedback (GRF):** Proposed by Mackie et al. [9], GRF uses an LLM to generate diverse text (keywords, essays, facts) relevant to the query. The generated text is treated as a feedback set from which expansion terms are extracted using a probabilistic model.

Query Rewriting transforms the original query into a more effective form without necessarily adding new terms. The Rewrite-Retrieve-Read framework [10] employs a trainable language model to reformulate queries before retrieval, optimizing the rewriter using reinforcement learning signals from the downstream reader model.

These query enhancement techniques operate as modular layers that can be added to existing RAG systems without modifying the retriever index or the generator, making them particularly attractive for practical deployment.

2.1.5 Arabic Language Processing Challenges

The Arabic language presents several characteristics that compound the retrieval challenges described in the preceding sections.

Morphological Richness. Arabic employs a root-and-pattern morphological system in which words are derived from typically tri-consonantal roots through templatic patterns and extensive affixation. A single root can yield dozens of surface forms through prefixes, suffixes, infixes, and clitics. For example, the root *k-t-b* (- - , “to write”) generates forms such as *kitaab* (book), *kaatib* (writer), *maktaba* (library), and *maktuub* (written). This morphological productivity creates a severe vocabulary mismatch problem, as queries and relevant documents may use different inflected forms of the same root.

Diglossia. Arabic exhibits a diglossic situation in which Modern Standard Arabic (MSA) is used in formal writing, education, and media, while various regional dialects (Egyptian, Levantine, Gulf, Maghrebi, and others) are used in daily speech and informal written communication. Knowledge bases are predominantly composed of MSA text, whereas user queries—particularly in conversational settings—may be formulated in dialectal Arabic, creating a systematic gap between query and document vocabularies [11].

Orthographic Variations. Arabic script permits multiple orthographic representations for certain characters, leading to inconsistencies even within MSA. Common variations include the different forms of Hamza (, (Alif (, (and Ya/Alif Maqsura (, (.(These variations can cause both sparse and dense retrieval methods to treat orthographic variants as distinct terms.

Diacritical Marks. Arabic diacritics (*tashkeel*) disambiguate pronunciation and meaning but are typically omitted in modern texts. This omission increases lexical ambiguity, as identical consonantal skeletons may correspond to multiple words with different meanings.

These characteristics collectively create what may be termed a “morphological gap” between user queries and knowledge base documents, which is distinct from and often more severe than the vocabulary mismatch observed in English information retrieval. Research by Alsubhi et al. [12] has demonstrated that these linguistic properties significantly affect RAG pipeline component selection, with sentence-aware chunking and multilingual embedding models outperforming simpler alternatives for Arabic text.

2.2 Mathematical Models

This section presents the mathematical formulations of the retrieval scoring functions and evaluation metrics employed in this thesis.

2.2.1 BM25 Scoring Function

The BM25 ranking function [4] computes the relevance score of a document d with respect to a query $q = \{q_1, q_2, \dots, q_n\}$ consisting of n terms:

$$\text{BM25}(q, d) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, d) \cdot (k_1 + 1)}{f(q_i, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \quad (2.4)$$

where $f(q_i, d)$ is the term frequency of query term q_i in document d , $|d|$ is the length of document d in tokens, avgdl is the average document length across the collection, k_1 controls the term frequency saturation (typically $k_1 = 1.2$), and b controls the degree of document length normalization (typically $b = 0.75$).

The Inverse Document Frequency (IDF) component is computed as:

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right) \quad (2.5)$$

where N is the total number of documents in the collection and $n(q_i)$ is the number of documents containing term q_i . The IDF assigns higher weight to rarer terms, which are generally more discriminative for retrieval.

2.2.2 Dense Retrieval and Cosine Similarity

In dense retrieval, queries and documents are encoded into dense vector representations by neural encoder models. Given a query encoder E_q and a document encoder E_d , the relevance score between query q and document d is computed as the cosine similarity between their embedding vectors:

$$\text{sim}(q, d) = \cos(\mathbf{e}_q, \mathbf{e}_d) = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\| \|\mathbf{e}_d\|} \quad (2.6)$$

where $\mathbf{e}_q = E_q(q)$ and $\mathbf{e}_d = E_d(d)$ are the embedding vectors for the query and document respectively, $\mathbf{e}_q \cdot \mathbf{e}_d$ denotes the dot product, and $\|\cdot\|$ denotes the Euclidean norm. When embedding vectors are L_2 -normalized, cosine similarity is equivalent to the dot product, and retrieval can be performed efficiently using Maximum Inner Product Search (MIPS) algorithms.

2.2.3 Evaluation Metrics

Three standard information retrieval metrics are used throughout this thesis to evaluate retrieval performance.

2.2.3.1 Recall at k (Recall@ k)

Recall at k measures the proportion of all relevant documents that appear within the top- k retrieved results:

$$\text{Recall@}k = \frac{|\{\text{relevant documents}\} \cap \{\text{top-}k \text{ retrieved}\}|}{|\{\text{relevant documents}\}|} \quad (2.7)$$

$\text{Recall@}k$ captures the retriever's ability to surface relevant documents within the top- k positions. In RAG systems, $\text{Recall@}k$ at the retrieval stage directly bounds the information available to the generator.

2.2.3.2 Normalized Discounted Cumulative Gain at k (NDCG@ k)

Normalized Discounted Cumulative Gain (NDCG) measures ranking quality by considering both the relevance of retrieved documents and their positions. Documents at higher ranks contribute more to the score, reflecting the intuition that users examine top-ranked results more carefully.

The Discounted Cumulative Gain at position k is defined as:

$$\text{DCG@}k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)} \quad (2.8)$$

where rel_i is the relevance grade of the document at position i . The ideal DCG ($\text{IDCG@}k$) is computed by sorting all relevant documents by decreasing relevance. $\text{NDCG@}k$ is the ratio:

$$\text{NDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k} \quad (2.9)$$

$\text{NDCG@}k$ ranges from 0 to 1, where 1 indicates a perfect ranking.

2.2.3.3 Mean Reciprocal Rank (MRR)

Mean Reciprocal Rank measures the average of the reciprocal ranks of the first relevant document across a set of queries Q :

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \quad (2.10)$$

where rank_i is the position of the first relevant document for the i -th query. MRR emphasizes the ability to place at least one relevant document at the top of the ranked list.

2.3 Models Used in Experiments

This section describes all language models employed in the experimental work of this thesis. The models were selected based on three primary criteria: (1) strong Arabic language support, either through specialized Arabic training or robust multilingual coverage; (2) compatibility with consumer-grade GPU hardware, specifically the NVIDIA T4 (15 GB VRAM) available through Google Colab and the NVIDIA A100 (40 GB VRAM); and (3) open-source availability with permissive licensing to ensure reproducibility [2]. Models are organized into Arabic-specialized, multilingual, and experimental categories. Additionally, the retrieval models used for passage ranking are described.

2.3.1 Arabic-Specialized Models

2.3.1.1 Falcon-H1-Arabic-3B

Falcon-H1-Arabic-3B is a 3.15 billion parameter language model developed by the Technology Innovation Institute (TII), Abu Dhabi [13]. Unlike conventional Transformer-based LLMs, Falcon-H1 employs a hybrid Mamba2-Transformer architecture that combines State Space Model (SSM) layers [14] with standard multi-head attention layers and SwiGLU activation functions. The model contains 32 layers with a hidden size of 2,560, utilizing 32 Mamba heads alongside 10 attention heads with Grouped Query Attention (GQA, 2 key-value heads).

The hybrid architecture was designed to combine the linear-time sequence processing efficiency of SSMS with the strong in-context learning capabilities of attention mechanisms. The model supports a context length of 128K tokens and was purpose-built for Arabic, listing it as a primary language alongside 17 additional languages.

On the Open Arabic LLM Leaderboard (OALL v2), Falcon-H1-Arabic-3B achieved an average score of approximately 62%, roughly 10 points ahead of competing models at the same parameter count. The model was released in May 2025 under an open license. In FP16 precision, the model requires approximately 10–11 GB of VRAM including SSM state buffers, fitting on both T4 and A100 GPUs without quantization.

2.3.1.2 Jais-2-8B

Jais-2-8B-Chat is an 8.09 billion parameter Arabic-centric language model developed collaboratively by Inception (G42), Mohamed bin Zayed University of Artificial Intelligence (MBZUAI), and Cerebras [15]. The model employs a standard decoder-only Transformer architecture with Rotary Position Embeddings (RoPE), Squared-ReLU activation, and LayerNorm. It contains 32 layers with a hidden size of 3,328 and 26 attention heads.

A distinguishing feature of Jais-2 is its Arabic-centric vocabulary of 150,272 tokens, trained from scratch to optimize Arabic tokenization efficiency. The model was pre-trained on 2.6 trillion tokens—6.6 times more data than its predecessor Jais-1—with substantial Arabic and English representation. The instruction-tuned variant was refined through a three-stage alignment pipeline comprising Supervised Fine-Tuning (SFT) on approximately 4 million Arabic and 10 million English prompt-response pairs, followed by DPO, and finally Group Relative Policy Optimization (GRPO).

On the AraGen-12-24 benchmark, Jais-2-8B-Chat achieved an overall 3C3H score of 58.64%, outperforming Fanar-1-9B and ALLaM-7B. The model was released in December 2024 under Apache 2.0 license (gated access). It requires BF16 precision due to Squared-ReLU activation overflow in FP16, using approximately 16.6 GB of VRAM on the A100.

2.3.1.3 ALLaM-7B

ALLaM-7B-Instruct-preview is a 7 billion parameter Arabic-English language model developed by the National Center for Artificial Intelligence (NCAI) at the Saudi Data and Artificial Intelligence Authority (SDAIA) [16]. Published at the International Conference on Learning Representations (ICLR) 2025, ALLaM employs the LlamaForCausalLM architecture with an expanded vocabulary of 64,000 tokens (doubled from Llama-2’s 32K to accommodate Arabic).

The model was trained from scratch in two phases: (1) a foundation phase on 4 trillion English tokens, followed by (2) continued pre-training on 1.2 trillion mixed Arabic-English tokens at a 45%/45%/10% Arabic/English/code ratio, totaling 5.2 trillion training tokens. The Arabic data comprised approximately 540 billion tokens, half of which were machine-translated from English sources. Instruction tuning was performed on 6 million samples, and alignment was achieved via DPO on approximately 245K preference pairs.

On the Arabic Massive Multitask Language Understanding (MMLU) benchmark, ALLaM achieved a 0-shot score of 67.78. The model was released in February 2025 under Apache 2.0 license. Notably, the publicly available version carries “preview” (alpha) status, indicating that it may not fully reflect the capabilities described in the published paper. The model requires approximately 14.5 GB of VRAM in FP16 on the A100.

2.3.1.4 SILMA Kashif-2B

SILMA Kashif-2B-Instruct is a 2 billion parameter language model developed by SILMA AI, an Arabic AI startup, with a specific focus on Arabic RAG applications [17]. The model was optimized for extractive question answering in Arabic, achieving a RAG Question Answering (RAGQA) benchmark score of 0.3575, which placed it at the top of the 3–9 billion parameter range on this benchmark.

The model supports a context length of 12K tokens and operates in FP16 precision, requiring approximately 4–5 GB of VRAM. Its small size makes it the most computationally

efficient model in the comparison, suitable for deployment on consumer-grade hardware without quantization.

2.3.2 Multilingual Models

2.3.2.1 Qwen 2.5 3B

Qwen 2.5-3B-Instruct is a 3.09 billion parameter multilingual language model developed by the Qwen Team at Alibaba Cloud [18]. The model employs a standard decoder-only Transformer architecture with GQA (16 query heads, 2 key-value heads), RMSNorm, SiLU activation, and a vocabulary of 151,936 tokens. It was pre-trained on approximately 18 trillion tokens spanning 29 or more languages, with Arabic included as a supported language.

Qwen 2.5-3B-Instruct was released in September 2024 under Apache 2.0 license. The model requires approximately 6 GB of VRAM in FP16, fitting comfortably on the T4 GPU. In the context of this thesis, Qwen 2.5-3B served as the reference model for the first Query2Doc experiment, establishing the initial query enhancement baseline.

2.3.2.2 Qwen 2.5 7B

Qwen 2.5-7B-Instruct is the 7 billion parameter variant in the same model family [18]. It shares the same architecture and tokenizer as the 3B variant but with a wider hidden dimension and more attention heads, resulting in stronger performance across multilingual benchmarks (MMLU: 74.2). The model requires 4-bit quantization to fit on the T4 GPU but operates in full FP16 precision on the A100. Pre-quantized versions are available from both the Qwen team and the Unsloth community.

2.3.2.3 Qwen3-4B

Qwen3-4B is a 4.0 billion parameter (3.6 billion non-embedding) next-generation language model from the Qwen Team [19]. Released in May 2025, it represents a significant

architectural evolution over Qwen 2.5: the number of supported languages was expanded from 29 to 119 (including eight Arabic dialects: MSA, Najdi, Levantine, Egyptian, Moroccan, Mesopotamian, Ta'izzi-Adeni, and Tunisian), training data was doubled to approximately 36 trillion tokens, and attention capacity was increased to 32 query heads with 8 key-value heads.

A notable feature of Qwen3 is its built-in *thinking mode*, which generates internal reasoning within special tags before producing the final answer. For query expansion tasks, this thinking mode must be explicitly disabled to prevent reasoning artifacts from contaminating the generated pseudo-documents. The model also requires sampling-based decoding; greedy decoding causes performance degradation and repetitive output.

Qwen3-4B matches or exceeds Qwen 2.5-7B on many benchmarks despite having fewer parameters, achieving an MMMLU (multilingual MMLU) score of 71.42. It operates in FP16 precision requiring approximately 8.5 GB of VRAM.

2.3.2.4 Qwen3-8B

Qwen3-8B is the 8 billion parameter variant of the Qwen3 family [19], sharing the same architectural innovations as Qwen3-4B including 119-language support, thinking mode, and 36 trillion training tokens. As the larger variant, it provides stronger language understanding and generation capabilities. Like Qwen3-4B, thinking tags must be stripped from generated output when used for query expansion. The model operates in FP16 on the A100 GPU.

2.3.2.5 Gemma 3 4B-IT

Gemma 3 4B-IT is a 4 billion parameter instruction-tuned language model developed by Google DeepMind [20]. Released in March 2025, it supports over 140 languages and features a context length of 128K tokens. The model employs a standard dense Transformer architecture with multimodal capabilities (vision and text), though only the text modality was used in this thesis.

Despite its broad language coverage, Gemma 3 exhibited the weakest Arabic performance among the tested models, scoring approximately 10 points below Falcon-H1-Arabic-3B on the OALL benchmark. It serves as a lower-bound comparison point in the model evaluation, representing a general-purpose multilingual model without Arabic-specific optimization.

2.3.2.6 Aya ExpansE 8B

Aya ExpansE 8B is an 8 billion parameter multilingual language model developed by Cohere Labs (CohereForAI), purpose-built for 101 languages with Arabic as one of the explicitly optimized languages [21]. The model was trained using a data-centric approach that emphasizes high-quality multilingual training data and achieves an 83.9% win rate against Llama-3.1-8B-Instruct in multilingual evaluations.

Aya ExpansE employs a standard Transformer architecture and was loaded with 4-bit NF4 quantization in the experiments due to its 8 billion parameter size. Despite the quantization, it demonstrated strong Arabic generation quality, suggesting that architecture and training data quality outweigh precision for this task. The model is distributed under a CC-BY-NC-4.0 (non-commercial) license.

2.3.3 Experimental Models

2.3.3.1 GPT-OSS-20B

GPT-OSS-20B is a 20.91 billion parameter language model released by OpenAI in August 2025 [22]—the organization’s first open-source model. It employs a Mixture-of-Experts (MoE) Transformer architecture with 32 experts and top-4 routing per token, resulting in only 3.61 billion active parameters per forward pass despite the large total parameter count. The architecture features 24 layers with alternating sliding-window and full attention patterns, GQA (64 query heads, 8 key-value heads), and a vocabulary of 201,088 tokens using the o200k_harmony byte-pair encoding tokenizer.

The model’s MoE expert weights are natively quantized to MXFP4 (Microscaling FP4) precision during training, achieving an effective 4.25 bits per parameter. This quantization-aware training means that the published benchmarks reflect MXFP4 performance rather than post-training quantization losses.

On the Arabic MMMLU benchmark, GPT-OSS-20B scored 76.3 (high reasoning effort), while on the community ILMAAM Arabic benchmark it achieved approximately 58%. The model’s training data is predominantly English, with no disclosed Arabic data proportions, and the model card explicitly warns that “GPT-OSS may not perform as well in other languages or may prefer English responses.” This model was included in the comparison to evaluate whether a powerful English-dominant MoE architecture can perform effective Arabic query expansion without language-specific training.

2.3.4 Retrieval Models

2.3.4.1 Multilingual Dense Passage Retriever (mDPR)

The Multilingual Dense Passage Retriever (mDPR) [5] is a bi-encoder dense retrieval model based on multilingual BERT. In this thesis, the pre-built mDPR index for MIRACL Arabic was used, which encodes the 2.1 million Arabic Wikipedia passages into dense vectors. mDPR was intentionally selected as the dense baseline because it was *not* fine-tuned on MIRACL training data, unlike models such as BGE-M3 and E5 which were partially trained on MIRACL. This choice provides a more realistic evaluation of query enhancement effectiveness, as improvements cannot be attributed to the retriever having memorized the evaluation queries.

2.3.4.2 BM25S

BM25S [23] is a Python-native implementation of the BM25 algorithm that eliminates the Java dependencies required by traditional implementations such as Pyserini. In this thesis, BM25S was used with default parameters ($k_1 = 1.5$, $b = 0.75$) over the MIRACL Arabic passage collection. BM25S was selected over Pyserini for its simpler

deployment pipeline and faster iteration speed, achieving 96% of the Pyserini BM25 baseline performance on the MIRACL Arabic evaluation set.

2.3.5 Model Comparison Summary

Table 2.1 presents a comparative summary of all language models used in the experiments. The models span a range of parameter counts from 2 billion to 20.9 billion, encompassing Arabic-specialized, multilingual, and experimental architectures.

Table 2.1: Summary of language models used in the experiments.

Model	Developer	Params	Architecture	Arabic Focus	License
<i>Arabic-Specialized</i>					
Falcon-H1-3B	TII	3.15B	Hybrid Mamba2-Trans.	Primary	Open
Jais-2-8B	MBZUAI/Inception	8.09B	Decoder-only Trans.	Primary	Apache 2.0
ALLaM-7B	SDAIA	7.0B	LlamaForCausalLM	Primary	Apache 2.0
SILMA Kashif-2B	SILMA AI	2.0B	Decoder-only Trans.	Primary	Open
<i>Multilingual</i>					
Qwen 2.5 3B	Alibaba	3.09B	Decoder-only Trans.	29+ langs	Apache 2.0
Qwen 2.5 7B	Alibaba	7.0B	Decoder-only Trans.	29+ langs	Apache 2.0
Qwen3-4B	Alibaba	4.0B	Decoder-only Trans.	119 langs	Apache 2.0
Qwen3-8B	Alibaba	8.0B	Decoder-only Trans.	119 langs	Apache 2.0
Gemma 3 4B	Google DeepMind	4.0B	Decoder-only Trans.	140+ langs	Gemma TO
Aya Expanse 8B	Cohere Labs	8.0B	Decoder-only Trans.	101 langs	CC-BY-NC
<i>Experimental</i>					
GPT-OSS-20B	OpenAI	20.9B*	MoE Transformer	English-dom.	Apache 2.0

*3.61B active parameters per token due to MoE routing (32 experts, top-4).

2.4 Related Work

This section reviews the published literature on query enhancement for information retrieval and RAG systems, organized chronologically and thematically. The review progresses from foundational work on generative query expansion to modern approaches using smaller LLMs, and concludes with Arabic-specific information retrieval research.

2.4.1 Foundational Query Enhancement Research (2022–2023)

The foundational work on LLM-based query enhancement was conducted using large proprietary models. Gao et al. [8] introduced HyDE, which demonstrated that an instruction-following LLM (InstructGPT, 175B parameters) could generate hypothetical documents for zero-shot dense retrieval, achieving performance comparable to supervised models without requiring any relevance labels. HyDE’s key insight was that document-to-document similarity is more reliable than query-to-document similarity, and that even hallucinated documents capture useful relevance patterns when encoded through a dense bottleneck.

Wang et al. [6] extended this concept with Query2Doc, which generates pseudo-documents and concatenates them with the original query rather than replacing it. Using `text-davinci-003` (175B parameters) with few-shot prompting, Query2Doc achieved 3–15% improvement over BM25 and made sparse retrieval competitive with state-of-the-art dense methods. Critically, the authors tested smaller models (OPT-1.3B and OPT-6.7B) and found them insufficient for quality query expansion—a finding that has since been challenged by newer, more capable small models. For sparse retrieval, the paper introduced query repetition (repeating the original query 5 times before concatenation) to prevent term dilution.

Mackie et al. [9] proposed Generative Relevance Feedback (GRF), which uses GPT-3 to generate diverse content types (keywords, essays, facts, news articles) as feedback for query expansion. GRF achieved 5–19% improvement in MAP and 17–24% in NDCG@10 over BM25+RM3 baselines. In subsequent work [24], the authors extended GRF to dense and learned sparse retrieval models, demonstrating approximately 9–10% improvement in NDCG@20. A key finding was that GRF and traditional pseudo-relevance feedback are complementary: their fusion achieved the best Recall@1000 in 17 out of 18 experiments.

Ma et al. [10] introduced the Rewrite-Retrieve-Read framework, which employs a trainable T5-large model as a query rewriter optimized via reinforcement learning from the downstream reader’s performance. This work demonstrated that small, trainable models can effectively improve the performance of large, frozen LLMs by operating at the query formulation stage.

A common characteristic of all foundational work is the reliance on 175B-parameter models (the GPT-3 family) for query expansion, with smaller models either untested or found inadequate.

2.4.2 Modern LLM-Based Query Enhancement (2024–2025)

The period from 2024 to 2025 saw significant progress in demonstrating that moderately-sized open-source LLMs (7–8B parameters) can perform effective query expansion, overturning the assumption that 175B-scale models are necessary.

Lei et al. [25] proposed Corpus-Steered Query Expansion (CSQE), which uses the target corpus to guide LLM-generated expansions. The CSQE pipeline operates in two stages. First, a first-pass sparse retrieval step retrieves a small set of top- k documents from the target corpus using the original query. These retrieved documents are then provided to the LLM alongside the query, instructing it to extract and synthesise topically relevant vocabulary and context grounded in the actual corpus content. This corpus-grounded expansion is combined with a blind expansion (standard Query2Doc) to produce the final enriched query. The key motivation is that corpus grounding anchors the LLM’s output to attested Wikipedia vocabulary, preventing the hallucination of plausible but factually incorrect entities that characterises blind generation for niche or ambiguous queries. Lei et al. reported that even a 7B-parameter model achieves a 30% improvement in mAP over BM25 on English benchmarks [25].

Zhang et al. [26] introduced MUGI (Multi-Granularity Indexing), which combines LLM-generated query expansions with a small bi-encoder retriever. The approach achieved up to 18% NDCG improvement with GPT-4-generated expansions, and demonstrated that even a 23-million-parameter bi-encoder can benefit from high-quality expansions.

Several additional studies confirmed the viability of smaller models for query enhancement. Zhang et al. [27] demonstrated personalized query expansion with GPT-4o-mini (approximately 8B parameters), achieving 10.5% Recall@5 improvement. Yoon et al. [28] tested Llama-3.1-8B and Mistral-7B for query expansion, finding that all tested LLMs improved retrieval performance and that 8B models approached GPT-4-level effectiveness. Xia et al. [29]

proposed Knowledge-Augmented Retrieval (KAR), showing that structured knowledge from Llama-3.1-8B was sufficient to achieve 36 MRR points improvement over HyDE. Yang et al. [30] demonstrated aligned query expansion with Mistral-7B, achieving 17.8% accuracy improvement through alignment fine-tuning. Lei et al. [31] proposed ThinkQE, which leverages chain-of-thought reasoning in a Qwen-14B model to surpass GPT-4-based expansion methods.

Research on knowledge distillation further reduced the model size requirements. Young et al. [32] proposed GaQR, which distills query rewriting capabilities from GPT-3.5 into Llama2-7B via a three-phase pipeline (generation, verification, distillation), achieving 4–9x faster inference than chain-of-thought methods. Chan et al. [33] introduced RQ-RAG, which trains Llama2-7B to autonomously refine queries through rewriting, decomposition, and disambiguation, achieving state-of-the-art performance for 7B-parameter models on multi-hop QA tasks. Louis and Bhusal [34] further demonstrated that knowledge distillation from GPT-3.5 to Flan-T5-base (approximately 250M parameters) can produce effective query augmentation, suggesting a path toward extremely efficient deployment.

Zhang et al. [35] addressed query robustness with QE-RAG, a benchmark specifically designed to evaluate RAG system performance under query entry errors (typos, misspellings, visual similarities). Their findings demonstrated that query enhancement techniques significantly improve retrieval robustness against noisy inputs—a characteristic analogous to the morphological and orthographic variations present in Arabic queries.

Collectively, this body of work demonstrates that the model size requirement for effective query expansion has decreased dramatically: from 175B parameters in 2022–2023 to 7–8B parameters in 2024–2025, with further compression possible via knowledge distillation. However, a critical gap remains: *none of these studies tested models smaller than 7B for zero-shot query expansion on Arabic text.*

2.4.3 Arabic Information Retrieval and RAG

Research on Arabic-specific RAG systems has begun to establish baselines, though work on query enhancement for Arabic remains scarce.

Zhang et al. [36] introduced MIRACL (Multilingual Information Retrieval Across a Continuum of Languages), a large-scale multilingual retrieval benchmark covering 18 languages including Arabic. The Arabic subset contains over 2.1 million Wikipedia passages with human-annotated relevance judgments for 2,896 development queries and 799 test queries, making it the primary evaluation resource for Arabic retrieval research.

Alsubhi et al. [12] conducted the first systematic analysis of RAG pipeline components for Arabic. Testing chunking strategies, embedding models, rerankers, and generators, they found that sentence-aware chunking outperformed fixed-size and semantic chunking for Arabic text, and that multilingual embedding models (BGE-M3, multilingual E5-large) outperformed Arabic-specific models for retrieval. They also identified Aya-8B as a superior generator for semantically dense Arabic content compared to StableLM-1.6B.

El-Beltagy and Abdallah [11] explored RAG applications for Arabic, benchmarking twelve embedding models and five LLMs. Their work revealed that E5 and BGE models exhibited remarkable robustness to Arabic dialectal variations in queries, and that open-source models (Llama 3, Mistral 7B) performed comparably to GPT-3.5 Turbo for Arabic text generation in RAG contexts.

These studies establish that Arabic RAG systems can achieve reasonable performance with existing multilingual tools. However, they focus primarily on the retrieval and generation components of the pipeline. The query formulation stage—specifically, whether LLM-based query enhancement techniques developed for English transfer effectively to Arabic—remains uninvestigated.

2.4.4 Research Gap

The literature review reveals a clear gap at the intersection of two well-studied areas:

- a. **LLM-based query enhancement** has been extensively studied for English, with recent work demonstrating that 7–8B parameter models are sufficient for effective query expansion (Section 2.4.2). However, none of these studies evaluate Arabic or other morphologically rich languages.

- b. **Arabic RAG systems** have established retrieval and generation baselines (Section 2.4.3), but query enhancement techniques—which operate upstream of both retrieval and generation—have not been investigated for Arabic.

Specifically, the following questions remain unanswered:

- Can modern open-source LLMs (2–8B parameters) perform effective zero-shot query expansion for Arabic information retrieval?
- How do Arabic-specialized models compare against multilingual models for this task?
- Does the Query2Doc technique, originally validated with 175B-parameter models on English text, transfer to Arabic with smaller models?
- What model characteristics (parameter count, architecture, Arabic training data volume, vocabulary design) most influence query expansion quality for Arabic?

A further gap exists in the application of corpus-steered expansion to non-English retrieval. The original CSQE work [25] was evaluated exclusively on English benchmarks using English-language models; whether corpus grounding provides the same benefits for Arabic—where BM25 homonym sensitivity may corrupt the first-pass retrieval and misdirect the expansion—has not been investigated. Additionally, the interaction between corpus-steered expansion and hybrid BM25–Dense fusion has not been studied: specifically, whether applying query expansion asymmetrically to only one retriever in a hybrid system can outperform applying it to both is an open question with practical implications for retrieval pipeline design.

This thesis addresses these questions through a systematic evaluation of eleven open-source language models for Query2Doc-based query expansion on the MIRACL Arabic benchmark, as described in the following chapter.

2.5 Chapter Summary

This chapter established the theoretical and empirical foundations for the research presented in this thesis. The key points are summarized as follows:

- RAG systems address the hallucination and knowledge staleness limitations of LLMs by incorporating external knowledge retrieval (Section 2.1.2), but their effectiveness is bounded by retrieval quality.
- Information retrieval can be performed via sparse methods (BM25), dense methods (DPR), or their combination, each with distinct strengths (Section 2.1.3).
- Query enhancement techniques—particularly LLM-based generative expansion—offer a modular approach to improving retrieval effectiveness without modifying the index or generator (Section 2.1.4).
- Arabic presents unique challenges for information retrieval due to morphological richness, diglossia, and orthographic variations (Section 2.1.5).
- The mathematical models for BM25 scoring, cosine similarity, and the evaluation metrics Recall@ k , NDCG@ k , and MRR were formulated (Section 2.2).
- Eleven language models spanning Arabic-specialized, multilingual, and experimental architectures were described for use in the comparative evaluation (Section 2.3).
- Recent literature demonstrates that 7–8B parameter models are sufficient for English query expansion, but no study has evaluated this approach for Arabic (Section 2.4).
- A clear research gap exists at the intersection of query enhancement techniques and Arabic information retrieval, which this thesis addresses through systematic model comparison on the MIRACL benchmark (Section 2.4.4).

The following chapter presents the methodology employed to address this research gap, describing the experimental design, dataset, evaluation pipeline, and the specific configurations used for each model.

Chapter 3

Methodology

This chapter presents the methodology employed throughout this research. The experimental work was conducted in a systematic, phased approach: establishing retrieval baselines (Section 3.2), performing error analysis to identify failure patterns (Section 3.3), implementing and adapting the Query2Doc query enhancement technique (Section 3.4), and conducting a comprehensive model comparison across **ten open-source** Large Language Models (Section 3.5). The chapter then presents the query repetition methodology for resolving BM25 term-dilution (Section 3.6), the hybrid BM25–Dense fusion methodology (Section 3.7), the Corpus-Steered Query Expansion (CSQE) pipeline including its component ablation design and retriever-specific application strategy (Section 3.8), and the per-query error analysis methodology (Section 3.9). All experimental setup details, including dataset characteristics and hardware configurations, are described in Section 3.1. The results corresponding to each methodological phase are presented in Chapter 4.

3.1 Dataset and Experimental Setup

3.1.1 Dataset: MIRACL Arabic

All experiments in this thesis were conducted on the Arabic subset of the Multilingual Information Retrieval Across a Continuum of Languages (MIRACL) benchmark [36]. MIRACL was selected as the primary dataset for three reasons: (1) it is retrieval-focused with human-annotated relevance judgements, (2) it contains natural query-document mismatch characteristic of real-world Arabic information needs, and (3) it is the standard benchmark for multilingual retrieval research, enabling comparison with published results.

The MIRACL Arabic corpus consists of 2,061,414 passages derived from 656,982 Arabic Wikipedia articles. Each passage is identified by a document identifier in the format $X\#Y$, where X denotes the article and Y the passage number within that article. The development set, which was used for all experiments reported in this thesis, contains 2,896 queries with 29,197 human-annotated relevance judgements. Each judgement assigns a relevance grade ranging from 0 (not relevant) to 3 (highly relevant), provided by native Arabic speakers. The corpus is written exclusively in Modern Standard Arabic (MSA); consequently, the evaluation of dialectal Arabic queries falls outside the scope of this work.

The dataset was accessed directly from HuggingFace¹ without local download, as both the corpus and pre-built retrieval indices were available through the Pyserini toolkit [37].

3.1.2 Hardware and Software Environment

All experiments were conducted on Google Colab, utilising two GPU configurations depending on availability and model requirements:

- **NVIDIA T4 (15 GB VRAM):** Used for baseline experiments and models with fewer than 4 billion parameters in FP16 precision.
- **NVIDIA A100 (40 GB VRAM):** Used for models requiring 4-bit quantisation or exceeding 7 billion parameters, and for batch sizes larger than 8.

The software environment consisted of Python 3.10, PyTorch 2.0+, the Hugging Face Transformers library (v4.30+), Pyserini (v0.22.1) for index access, and FAISS for dense vector search. For models exceeding GPU memory in native precision, 4-bit NormalFloat (NF4) quantisation was applied using the `bitsandbytes` library [38], following the QLoRA quantisation scheme.

¹<https://huggingface.co/datasets/miracl/miracl>

3.1.3 Evaluation Metrics

Three standard information retrieval metrics were employed for evaluation, as defined in Section 2.2:

- **NDCG@10** (Equation 2.9): Measures the quality of ranking within the top-10 retrieved passages. This was treated as the primary metric throughout all experiments.
- **Recall@10** (Equation 2.7): Measures the proportion of all relevant passages that appear in the top-10 results.
- **MRR** (Equation 2.10): Measures how quickly the first relevant passage is retrieved.

Additionally, Recall@100 was computed to assess overall retrieval coverage. All metrics were calculated using the `pytrec_eval` library over the full development set of 2,896 queries.

3.1.4 Experimental Pipeline Overview

The experimental pipeline follows a two-phase architecture, as illustrated in Figure 3.1. In the first phase (query enhancement), each query from the MIRACL development set is processed by a query enhancement module, which may leave the query unchanged (baseline) or augment it using an LLM-generated pseudo-document. In the second phase (retrieval and evaluation), the enhanced query is passed to a retrieval system, and the top-100 results are evaluated against the MIRACL relevance judgements.

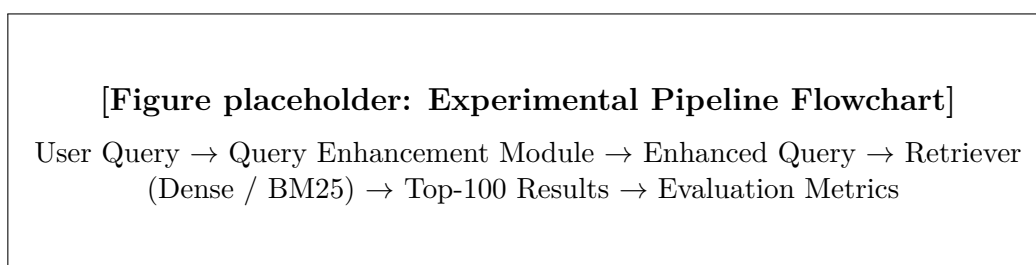


Figure 3.1: Overview of the experimental pipeline. The query enhancement module is the variable component across experiments; the retrieval and evaluation components remain fixed.

A key design decision was to separate query generation from retrieval evaluation into two independent notebooks. The query enhancement phase (which requires GPU for LLM inference) saves enhanced queries to disk as serialised Python objects. The evaluation phase then loads these pre-generated queries and performs retrieval and metric computation. This two-notebook workflow enabled efficient reuse of enhanced queries across multiple retrieval configurations (Dense and BM25) without regenerating them.

3.2 Baseline Implementation

Two retrieval baselines were established to provide reference performance levels against which all query enhancement experiments would be compared: a dense retrieval baseline using mDPR and a sparse retrieval baseline using BM25S. Both baselines used an `IdentityEnhancer` that returned queries unchanged.

3.2.1 Dense Retrieval Baseline (mDPR)

The dense retrieval baseline employed Multilingual Dense Passage Retrieval (mDPR), described in Section 2.3.4.1. The mDPR encoder (`castorini/mdpr-tied-pft-msmarco`) was loaded on GPU, and queries were encoded in batches of 64. The pre-built FAISS index for MIRACL Arabic (5.47 GB, 768-dimensional embeddings) was accessed via Pyserini. For each query, the top-100 passages were retrieved using cosine similarity.

The decision to use mDPR rather than stronger retrieval models such as BGE-M3 (described in Section 2.1.3.2) was deliberate. mDPR was pre-trained on MS MARCO (English) and was *not* fine-tuned on the MIRACL Arabic corpus. This intentionally “weaker” baseline was chosen to provide greater headroom for demonstrating the impact of query enhancement techniques. Stronger models that were trained directly on MIRACL would leave less room for measurable improvement.

3.2.2 Sparse Retrieval Baseline (BM25S)

The sparse retrieval baseline employed BM25S [23], a pure-Python implementation of the BM25 algorithm (described in Section 2.1.3.1). BM25S was selected over the Pyserini-based BM25 implementation after encountering a blocking dependency conflict between Java 21 and Java 11 in the Google Colab environment. The BM25S library eliminates all Java dependencies while maintaining competitive performance.

The BM25S index was built from the MIRACL Arabic corpus with the following configuration:

- **Tokenisation:** Arabic stemming using PyStemmer with NLTK Arabic stopwords (245+ words).
- **BM25 variant:** Lucene-style scoring with parameters $k_1 = 0.9$ and $b = 0.4$.
- **Index storage:** Google Drive (~0.5 GB), loaded via symbolic links in Colab.

This configuration achieved 96% of the official MIRACL Pyserini baseline performance, with the 4% difference deemed acceptable given the benefits of a pure-Python pipeline that integrates directly with the query enhancement modules.

3.2.3 Baseline Comparison Rationale

Dense and sparse retrieval were tested *separately* rather than combined into a hybrid system. This decision, confirmed during the project planning phase, was made to provide clearer insight into where query enhancement improvements originate. As discussed in Section 2.1.3, dense retrieval captures semantic similarity while sparse retrieval relies on exact term matching. Testing both baselines independently enables analysis of whether a given query enhancement technique benefits semantic matching, lexical matching, or both.

3.3 Error Analysis Methodology

Before implementing query enhancement techniques, a systematic error analysis was conducted on the dense retrieval baseline (Experiment 001) to identify the primary failure patterns and guide technique selection.

3.3.1 Quantitative Analysis Framework

A quantitative analysis was performed over all 2,896 queries in the MIRACL Arabic development set. Queries were segmented into three performance categories based on their individual NDCG@10 scores:

- **Failed:** $\text{NDCG@10} < 0.3$
- **Mediocre:** $0.3 \leq \text{NDCG@10} < 0.7$
- **Successful:** $\text{NDCG@10} \geq 0.7$

For each query, the following features were computed:

- a. **Query length** (in tokens), measured after whitespace tokenisation.
- b. **Score gap** between the top-ranked passage and the second-ranked passage, as a proxy for retriever confidence.
- c. **First relevant document rank**, indicating how deep the retriever must search before finding a relevant passage.
- d. **Coverage at multiple cut-offs** (top-10, top-20, top-50, top-100), measuring the proportion of queries with at least one relevant passage at each depth.

The Pearson correlation coefficient was computed between query length and NDCG@10 to quantify the relationship between query verbosity and retrieval performance.

3.3.2 Query Length Bucketing

Queries were grouped into three length buckets to analyse the effect of query length on retrieval performance:

- **Short:** 1–3 tokens (5.1% of queries)
- **Medium:** 4–8 tokens (86.2% of queries)
- **Long:** 9+ tokens (8.8% of queries)

The mean NDCG@10 was computed for each bucket, and the performance gap between short and long queries was used to quantify the impact of information poverty on retrieval effectiveness.

3.3.3 Failed Query Inspection

The 20 worst-performing queries ($\text{NDCG@10} = 0.000$, indicating zero relevant passages in the top-10) were manually inspected to identify recurring linguistic patterns. Each query was categorised by question type (factoid, definition, temporal), the presence of named entities, diacritics, technical terminology, and spelling variations. These observations informed the selection of the first query enhancement technique.

3.4 Query2Doc Implementation

Based on the error analysis findings (presented in Section 4.2), Query2Doc [6] was selected as the first query enhancement technique. This section describes the Query2Doc approach, the modifications made for Arabic zero-shot application, and the implementation details.

3.4.1 Query2Doc Technique

Query2Doc, introduced by Wang et al. [6], uses an LLM to generate a pseudo-document—a short passage that attempts to answer the query. This pseudo-document is then concatenated with the original query to form an enhanced query. The rationale is that the pseudo-document introduces additional vocabulary, context, and semantic information that bridges the gap between the query and relevant passages.

As described in Section 2.1.4, Query2Doc falls within the category of LLM-based generative query enhancement techniques. It differs from Hypothetical Document Embeddings (HyDE) [8] in a critical way: while HyDE uses the generated document *alone* as the retrieval query, Query2Doc *concatenates* the generated document with the original query, preserving the original query terms.

3.4.2 Modifications from the Original Paper

Several modifications were made to adapt Query2Doc for this research context:

- a. **Zero-shot prompting:** The original Query2Doc paper employed few-shot prompting with 4 demonstration examples using GPT-3 (text-davinci-003, 175B parameters). In this work, zero-shot prompting was adopted exclusively. This decision was driven by resource constraints (small open-source models have limited context windows for few-shot examples) and the desire to evaluate model capabilities without task-specific demonstrations.
- b. **Small open-source models:** Rather than using proprietary API-based models, all experiments utilised open-source models with 2–8 billion parameters that can run on Google Colab free-tier GPUs. This approach ensures reproducibility and eliminates API costs.
- c. **Arabic-only generation:** The system prompt explicitly instructs the model to respond in Arabic only, whereas the original paper targeted English queries.

- d. **Retriever-specific concatenation:** For dense retrieval, simple concatenation was used: $q_{\text{enhanced}} = q \oplus d_{\text{pseudo}}$, where $\oplus$ denotes string concatenation. The original paper recommends repeating the query $n = 5$ times before concatenation for sparse retrieval to prevent term dilution, as discussed in Section 3.4.6.

3.4.3 LLM Configuration

The initial Query2Doc experiment used Qwen 2.5 3B Instruct (described in Section 2.3.2.1) as the generation model. The following system prompt was used across all experiments:

“You are asked to write a passage that answers the given query. Do not ask the user for further clarification. Respond in Arabic only.”

The generation parameters were configured as follows:

Table 3.1: LLM generation parameters for the initial Query2Doc experiment (Experiment 003).

Parameter	Value
Maximum new tokens	128
Temperature	0.7
Top- p (nucleus sampling)	0.9
Batch size	8
Precision	FP16

The maximum generation length of 128 tokens (~ 100 Arabic words) was selected as a balance between expansion quality and inference speed. Initial testing with 256 tokens showed diminishing returns in retrieval improvement while doubling generation time.

3.4.4 Batch Processing Optimisation

A critical engineering challenge was generation throughput. Naïve sequential processing (~ 10 seconds per query) would require approximately 8 hours for the full development set. Three optimisations were implemented to achieve practical runtimes:

- a. **Batch processing:** Multiple queries were tokenised with left-padding and processed simultaneously through the LLM, yielding an $8\times$ speedup.
- b. **Reduced token generation:** The maximum generation length was reduced from 256 to 128 tokens, yielding a $2\times$ speedup with negligible quality loss.
- c. **Inference optimisations:** The model was set to evaluation mode (`model.eval()`), half-precision (`float16`) was used, and gradient computation was disabled (`torch.no_grad()`).

The combined effect was a $16\times$ speedup, reducing the full-set generation time from ~ 8 hours to ~ 40 minutes on a T4 GPU. This enabled rapid iteration across multiple models and configurations.

3.4.5 Dense Retrieval with Query2Doc

For dense retrieval experiments, the enhanced query was formed by simple concatenation:

$$q_{\text{enhanced}} = q_{\text{original}} \oplus d_{\text{pseudo}} \quad (3.1)$$

The enhanced query was then encoded by the mDPR encoder and used for FAISS similarity search, following the same procedure as the baseline (Section 3.2.1). This approach leverages the semantic content of the pseudo-document to improve the query's embedding representation.

3.4.6 BM25 Retrieval with Query2Doc

For sparse retrieval, the same enhanced queries were evaluated against the BM25S index. Notably, the initial BM25 experiment (Experiment 004) used simple concatenation identical to the dense configuration, *without* implementing the query repetition strategy recommended by Wang et al. [6]. The original paper recommends:

$$q_{\text{enhanced}}^{\text{BM25}} = \underbrace{q \oplus q \oplus \dots \oplus q}_{n \text{ times}} \oplus d_{\text{pseudo}} \quad (3.2)$$

where $n = 5$. This repetition preserves the importance of original query terms in the BM25 scoring function (Equation 2.4), preventing the pseudo-document from diluting the term weights of the original query. The omission of this strategy in the initial experiment, and its subsequent impact, is analysed in Section 4.3.2.

3.5 Model Comparison Methodology

Following the initial Query2Doc results with Qwen 2.5 3B, a comprehensive model comparison was conducted to identify the optimal LLM for Arabic query expansion. Ten open-source models were evaluated using the identical Query2Doc pipeline.

3.5.1 Model Selection Criteria

Models were selected based on three criteria: (1) Arabic language support, either through dedicated Arabic training or strong multilingual capabilities; (2) parameter count within the 2–8 billion range to remain executable on Google Colab GPUs; and (3) open-source availability with permissive licences for research use. The selected models span three categories, all of which are described in detail in Section 2.3:

- **Arabic-specialised:** Falcon-H1-3B (Section 2.3.1.1), Jais-2-8B (Section 2.3.1.2), ALLaM-7B (Section 2.3.1.3), and SILMA Kashif-2B (Section 2.3.1.4).
- **Multilingual:** Qwen 2.5 3B (Section 2.3.2.1), Qwen 2.5 7B (Section 2.3.2.2), Qwen3-4B (Section 2.3.2.3), Qwen3-8B (Section 2.3.2.4), Gemma 3 4B (Section 2.3.2.5), and Aya Expanse 8B (Section 2.3.2.6).
- **Experimental:** GPT-OSS-20B (Section 2.3.3.1), a Mixture-of-Experts model included to test whether larger architectures provide benefits despite resource constraints.

Table 3.2 summarises the configuration used for each model.

Table 3.2: Model configurations for the Query2Doc comparison experiments. All models used `max_new_tokens` = 128 and `top_p` = 0.9 unless otherwise noted.

Model	Params	Precision	Temp.	Batch	GPU
Qwen 2.5 3B	3B	FP16	0.7	8	T4
SILMA 2B	2B	FP16	0.1	16	A100
Falcon-H1 3B	3.15B	BF16	0.1	1*	A100
Qwen3-4B	4B	FP16	0.7	32	A100
Gemma 3 4B	4B	FP16	0.1	16	A100
Qwen 2.5 7B	7B	FP16	0.1	16	A100
ALLaM 7B	7B	NF4	0.7	—	A100
Jais-2 8B	8B	BF16	0.1	8	A100
Qwen3-8B	8B	FP16	0.1	16	A100
Aya Expanse 8B	8B	NF4	0.1	8	A100
GPT-OSS 20B	20B	NF4	0.1	—	A100

*Forced to batch size 1 due to architecture-specific batching bug (see Section 3.5.4).

3.5.2 Standardised Evaluation Protocol

To ensure fair comparison, all models were evaluated using an identical protocol:

- Sanity check:** The first 5 queries were processed and the outputs were manually inspected to verify that the model (a) generates text in Arabic, (b) produces relevant content, and (c) does not exhibit degenerate behaviour (repetition, code output, or empty responses).
- Full generation:** All 2,896 queries were processed using the Query2Doc pipeline, and the enhanced queries were saved to disk.
- Dense evaluation:** Enhanced queries were evaluated against the mDPR dense retrieval baseline.
- BM25 evaluation:** The same enhanced queries were evaluated against the BM25S sparse retrieval baseline.

The work was divided between the two team members: one researcher tested Falcon-H1-3B, Jais-2-8B, ALLaM-7B, Qwen3-4B, and GPT-OSS-20B, while the other tested SILMA

Kashif-2B, Qwen 2.5-7B, Qwen3-8B, Gemma 3 4B, and Aya Expanse 8B. All experiments used the same codebase, system prompt, and evaluation pipeline to minimise confounding variables.

3.5.3 Temperature Selection

Temperature was not fixed at a single value across all models. Two considerations guided temperature selection:

- a. **Model-recommended values:** Some models specify recommended generation parameters. Falcon-H1 recommends temperature 0.1, noting that higher values “largely drop performance.” Qwen3 models require sampling (`do_sample=True`) to avoid infinite repetitions with greedy decoding.
- b. **Empirical testing:** For SILMA Kashif-2B, two temperature values were compared (0.7 and 0.1). Temperature 0.1 yielded a 2.5% improvement in NDCG@10 over 0.7, leading to its adoption as the default for subsequent experiments where no model-specific recommendation existed.

3.5.4 Model-Specific Technical Issues

Several models required specific engineering workarounds, documented here for reproducibility:

- a. **Falcon-H1-3B:** The hybrid Mamba2-Transformer architecture (described in Section 2.3.1.1) contains a batching bug in the attention mask extension logic. When left-padded batches are used, the 4D causal attention mask is not extended as new tokens are generated, causing a shape mismatch. The workaround was to force `batch_size=1`, resulting in significantly longer generation times but correct outputs.
- b. **Jais-2-8B:** This model requires BF16 precision rather than FP16 due to its Squared-ReLU activation function, which produces values that overflow FP16 range. Additionally, the `token_type_ids` parameter must be excluded from the model inputs, and the padding token must be explicitly set to the end-of-sequence token.

- c. **Qwen3-4B and Qwen3-8B:** The Qwen3 generation includes “thinking” tokens (internal chain-of-thought reasoning) that must be stripped from the output. The parameter `enable_thinking=False` was passed during generation. Greedy decoding must not be used, as it causes infinite repetition loops.
- d. **ALLaM-7B:** As a preview-stage model, ALLaM exhibited a sentencepiece tokenizer artefact where the Unicode character “ ” (lower one-eighth block) leaked into decoded outputs. This special character, used internally by the sentencepiece tokenizer to denote word boundaries, contaminated the pseudo-documents. The impact on retrieval performance is discussed in Section 4.4.3.
- e. **GPT-OSS-20B:** The Mixture-of-Experts architecture (32 experts, top-4 routing, described in Section 2.3.3.1) proved impractical through 4-bit quantisation, exhibiting generation speeds approximately 70× slower than dense models of comparable active parameter count. Furthermore, its Harmony chat format required a forced-final-channel prefix to suppress English-language reasoning.

3.5.5 Quantisation Strategy

Models exceeding the available GPU memory in native precision were quantised using 4-bit NormalFloat (NF4) quantisation via the `bitsandbytes` library [38]. The quantisation configuration was:

- **Quantisation type:** NF4 (NormalFloat 4-bit)
- **Compute dtype:** BF16 (for matrix multiplication)
- **Double quantisation:** Enabled (quantises the quantisation constants for further memory savings)

This configuration was applied to Jais-2-8B (on T4 only; BF16 was used on A100), ALLaM-7B, Aya Expanse 8B, and GPT-OSS-20B. Notably, **Aya Expanse 8B achieved the best overall performance despite being quantised to 4-bit**, suggesting that model architecture and training data quality are more significant factors than numerical precision for this task.

3.6 Query Repetition for Sparse Retrieval

The model comparison experiments (Section 4.4.2) revealed that single-pass Query2Doc ($n = 1$) degraded BM25 performance for six of nine models due to term dilution (Section 4.3.2.1). The original Query2Doc paper [6] recommended concatenating $n = 5$ pseudo-documents with equal weighting, but did not investigate the sensitivity of the repetition factor to model size or pseudo-document length. Two solution families were evaluated to address this limitation.

Fixed Repetition (Query2Doc-style). The original query is prepended n times before the pseudo-document, where n is a global hyperparameter applied identically to every query regardless of its length or the pseudo-document length. The enhanced query is assembled as:

$$q_{\text{rep}} = \underbrace{q \circ q \circ \dots \circ q}_{n \text{ times}} \circ d_1 \circ d_2 \circ \dots \circ d_k \quad (3.3)$$

where $\circ$ denotes string concatenation with a space separator and k is the number of pseudo-documents (for single-pass Query2Doc, $k = 1$). Fixed repetition was swept over $n \in \{1, 3, 5, 7, 10\}$.

Adaptive Repetition (MuGI-style). Zhang et al. [26] proposed computing n per-query from the ratio of pseudo-document length to query length, controlled by a parameter β :

$$n(q, d, \beta) = \max\left(1, \left\lfloor \frac{|d|}{|q| \cdot \beta} \right\rfloor\right) \quad (3.4)$$

where $|q|$ and $|d|$ are the token lengths of the query and pseudo-document respectively. Larger β yields fewer repetitions; smaller β yields more. Adaptive repetition was swept over $\beta \in \{2, 4, 6\}$.

Motivation for the Adaptive Variant. The optimal number of repetitions depends on the length ratio between query and pseudo-document. A 3-word query paired with a 200-token pseudo-document requires more repetition than a 15-word query paired with a 100-token pseudo-document. The MuGI per-query adaptation removes this length sensitivity and, as reported in Section 4.6, produced the best results for the two largest models (Aya Expanse 8B and Jais-2 8B), where $\beta = 2$ was the winning configuration.

Sweep Design. Nine models were evaluated across eight configurations ($n \in \{1, 3, 5, 7, 10\}$ and $\beta \in \{2, 4, 6\}$), yielding 72 BM25 evaluations. All pseudo-document expansions had been generated during the model comparison phase (Section 3.5); no new LLM inference was required. The stored expansion files (serialised Python objects) were loaded and combined with varying repetition factors, then evaluated using `pytrec_eval` on the full MIRACL Arabic development set (2,896 queries). The total sweep runtime was approximately 73 minutes on a Colab CPU instance.

3.7 Hybrid Retrieval Fusion

The baseline experiments (Section 4.1.1) confirmed that BM25 and mDPR exhibit complementary strengths: mDPR achieves superior ranking quality while BM25 provides broader lexical coverage. Hybrid retrieval fusion combines the ranked lists from both retrievers into a single ranking to exploit this complementarity. Two fusion methods, introduced in Section 2.1.3.3, were evaluated.

Reciprocal Rank Fusion (RRF). RRF (Equation 2.2) combines ranked lists by summing inverse rank scores with a smoothing constant k . Two values were tested: $k = 20$, following the standard practice established by Bruch et al. [7], and $k = 60$.

Convex Combination (CC). CC (Equation 2.3) computes a weighted sum of normalised retriever scores, where α controls the relative contribution of the dense retriever. Before combination, the scores from each retriever were min-max normalised to $[0, 1]$ on a

per-query basis to account for the different score scales of BM25 and mDPR. The weight α was swept over $\{0.1, 0.2, \dots, 0.9\}$.

Setup. Both BM25S and mDPR retrieved the top-100 candidates independently using the original (unenhanced) queries. Fusion and re-ranking were performed in Python without additional GPU inference. The resulting hybrid baseline establishes the performance ceiling that all subsequent query enhancement methods must surpass.

3.8 Corpus-Steered Query Expansion (CSQE)

Corpus-Steered Query Expansion (CSQE), introduced by Lei et al. [25] and described in Section 2.4.2, extends blind Query2Doc by grounding LLM-generated expansions in first-pass retrieved documents. This section describes the CSQE pipeline as adapted for Arabic retrieval, the component ablation design, and the retriever-specific application strategy.

3.8.1 The CSQE Pipeline

The CSQE pipeline distinguishes itself from standard (blind) Query2Doc through a two-stage design:

- a. **First-pass retrieval.** BM25S retrieves the top- $k = 5$ documents using the original query q .
- b. **Corpus-grounded expansion.** Each retrieved document d_i is concatenated with q and passed to the LLM with a CSQE-specific system prompt instructing the extraction of topically relevant Arabic vocabulary grounded in the retrieved passage.
- c. **Blind expansion.** The same query q is passed to the LLM without any retrieved context, following the standard Query2Doc procedure.

- d. **Expansion assembly.** The corpus-grounded and blind samples are combined with a repeated original query:

$$q_{\text{CSQE}} = \underbrace{q \circ q \circ \dots \circ q}_{\alpha \text{ times}} \circ c_1 \circ c_2 \circ b_1 \circ b_2 \quad (3.5)$$

where c_i denotes corpus-grounded expansion samples, b_i denotes blind expansion samples, and α is the query repetition factor (Equation 3.3).

The configuration used for the primary CSQE experiment was: $k = 5$ first-pass documents, 2 corpus-grounded samples, 2 blind samples, $\alpha = 4$, temperature 1.0, `max_new_tokens` = 128 per sample, using Aya Expanse 8B (Section 2.3.2.6) in BF16 precision on an A100 GPU.

Rationale for Combining Corpus and Blind Samples. Corpus-grounded samples provide attested Arabic vocabulary anchored to the actual Wikipedia passages retrieved in the first pass. Blind samples provide diverse answer-space vocabulary unconstrained by the corpus. The two sources are hypothesised to be complementary: corpus samples prevent hallucination of incorrect entities, while blind samples expand vocabulary coverage beyond the retrieved passages. This hypothesis is validated through the ablation study presented in Section 4.8.2.

3.8.2 Component Ablation Design

To isolate the individual contributions of corpus-grounded and blind expansion, three ablation variants were defined:

- **Experiment 013c (Corpus-only):** `num_corpus_samples` = 4, `num_blind_samples` = 0. This variant isolates the contribution of first-pass grounding by using only corpus-grounded expansions.
- **Experiment 013d (Blind-only):** `num_corpus_samples` = 0, `num_blind_samples` = 4. This variant isolates the contribution of blind generation, equivalent to a 4-sample Query2Doc with repetition.

- **Experiment 013 (Full CSQE):** `num_corpus_samples = 2, num_blind_samples = 2`.
The combined system as described above.

All three variants used $\alpha = 4$ and the same Aya Expanse 8B model. Additionally, a query repetition factor sweep ($\alpha \in \{1, 2, 3, 4\}$) was conducted on the full CSQE variant. This sweep was reconstructed from stored expansion files (serialised Python objects) without requiring new LLM inference—only the number of query preprends was varied before re-running BM25 evaluation.

3.8.3 Retriever-Specific Application

The CSQE-expanded query (q_{CSQE}) is substantially longer than the original query ($\sim 1,500$ characters vs. ~ 30 characters). Dense encoders such as mDPR (Section 2.3.4.1) were trained on short natural-language queries, raising the hypothesis that long expanded queries may degrade embedding quality. To test this hypothesis, three fusion configurations were defined:

- **Config A:** BM25 receives the CSQE-expanded query; Dense receives the original query. *Hypothesis:* BM25 benefits from vocabulary breadth in the long expansion, while Dense performs best with the short query it was trained on.
- **Config B:** BM25 receives the original query; Dense receives the CSQE-expanded query. *Hypothesis:* the semantic signal in the CSQE expansion improves dense encoding.
- **Config C:** Both BM25 and Dense receive the CSQE-expanded query. *Hypothesis:* maximum expansion applied uniformly should yield the highest performance.

For each configuration, both RRF ($k = 20$) and CC (α swept over $\{0.1, \dots, 0.9\}$) were evaluated. This design isolates the effect of retriever-specific query representation and determines whether asymmetric expansion outperforms symmetric application.

3.9 Per-Query Error Analysis

Aggregate metrics such as NDCG@10 can mask substantial per-query variation: a system with a high average may still regress on a significant minority of queries. A per-query error analysis was conducted to decompose the overall CSQE improvement and identify the conditions under which corpus-steered expansion helps or hinders retrieval.

Per-Query Metric Computation. The `pytrec_eval` library was used to compute NDCG@10 for each of the 2,896 MIRACL Arabic development queries under both the CSQE+Hybrid system and the Aya Blind BM25 $n = 1$ baseline. The per-query delta was defined as $\Delta_i = \text{NDCG@10}_{\text{CSQE},i} - \text{NDCG@10}_{\text{Blind},i}$.

Classification Thresholds. Three categories were defined based on the per-query scores and deltas:

- **Failure:** CSQE NDCG@10 < 0.1. Queries where the system fails to retrieve any relevant document in the top-10, regardless of the baseline performance.
- **Big win:** $\Delta_i > 0.3$. Queries where CSQE achieves a large improvement over the blind baseline.
- **Regression:** $\Delta_i < -0.1$. Queries where CSQE performs worse than the blind baseline by a non-trivial margin.

First-Pass Quality Split. To assess the role of BM25 first-pass quality in CSQE effectiveness, queries were split into two groups based on whether the BM25 first-pass retrieval ($k = 5$) returned at least one relevant document (any relevance judgement > 0 in the MIRACL qrels for the top-5 retrieved documents). This split enables analysis of whether CSQE gains are driven primarily by successful corpus grounding or by the blind expansion component.

Regression Classification. Regression queries ($\Delta_i < -0.1$) were manually inspected and classified into three types:

- **Type A (Strong BM25 hurt):** The BM25 baseline already achieved $\text{NDCG@10} \geq 0.3$ for the query; the CSQE expansion introduced off-topic vocabulary that diluted the effective query terms.
- **Type B (Poisoned first-pass):** The BM25 baseline achieved $\text{NDCG@10} < 0.1$ for the query; the first-pass retrieval returned irrelevant documents, causing the LLM to ground its expansion on incorrect content.
- **Type C (Partial BM25):** The BM25 baseline achieved NDCG@10 between 0.1 and 0.3; a mixed-quality scenario where the expansion may have helped or hurt depending on specific query characteristics.

Chapter 4

Results and Discussion

This chapter presents the results of all experiments conducted in this research, following the zigzag structure with the methodology described in Chapter 3. Section 4.1 reports the baseline retrieval performance. Section 4.2 presents the error analysis findings that guided technique selection. Section 4.3 reports the initial Query2Doc results with both dense and sparse retrieval. Section 4.4 presents the comprehensive model comparison across ten LLMs. Section 4.5 synthesises the key findings from the model comparison phase. The chapter then reports the expanded Phase 4 results: the BM25 query repetition sweep (Section 4.6), hybrid retrieval fusion (Section 4.7), Corpus-Steered Query Expansion (Section 4.8), CSQE combined with hybrid fusion (Section 4.9), and a per-query error analysis decomposing the sources of improvement and regression (Section 4.10).

4.1 Baseline Results and Comparison

The two retrieval baselines described in Section 3.2 were evaluated on the full MIRACL Arabic development set (2,896 queries). Table 4.1 presents the results.

Table 4.1: Baseline retrieval results on MIRACL Arabic dev set (2,896 queries). Bold values indicate the higher score for each metric.

Retriever	NDCG@10	Recall@10	Recall@100	MRR
mDPR (Dense)	0.4993	0.6156	0.8407	0.5328
BM25S (Sparse)	0.4621	0.5964	0.8577	0.4836

The mDPR dense baseline reproduced the official MIRACL results with less than 0.1% difference (NDCG@10 = 0.4993 vs. published 0.499), confirming the correctness of the implementation. The BM25S sparse baseline achieved 96% of the official Pyserini BM25

performance ($\text{NDCG@10} = 0.4621$ vs. target 0.481), with the 4% difference attributable to the use of BM25S rather than Pyserini’s Java-based implementation, as discussed in Section 3.2.2.

4.1.1 Complementary Strengths

The two baselines exhibited complementary performance characteristics. mDPR achieved higher NDCG@10 (+8.1%) and MRR (+10.2%), indicating superior ranking quality—relevant passages were placed higher in the result list. BM25S achieved higher Recall@100 (+2.0%), indicating broader retrieval coverage—more relevant passages were retrieved overall, though ranked less effectively.

This complementary behaviour aligns with the theoretical properties of dense and sparse retrieval discussed in Section 2.1.3: dense retrieval captures semantic similarity and ranks well, while sparse retrieval excels at broad lexical matching. This finding confirmed the decision to test query enhancement techniques on both retrievers independently.

4.2 Error Analysis Findings

The error analysis methodology described in Section 3.3 was applied to the dense baseline (Experiment 001). The findings guided the selection of the first query enhancement technique.

4.2.1 Overall Failure Rate

Table 4.2 presents the performance segmentation of all 2,896 queries.

A striking finding was that 39% of queries (1,130 out of 2,896) were classified as failed, with 192 queries achieving zero recall in the top-10 (no relevant passages retrieved). Only 5.1% of queries were classified as highly successful. This high failure rate established a clear opportunity for query enhancement to make a measurable impact.

Table 4.2: Performance segmentation of baseline queries by NDCG@10 score.

Category	Count	% of Total	Avg NDCG@10
Failed ($\text{NDCG@10} < 0.3$)	1,130	39.0%	0.158
Mediocre ($0.3 \leq \text{NDCG@10} < 0.7$)	1,618	55.9%	0.472
Successful ($\text{NDCG@10} \geq 0.7$)	148	5.1%	0.762

4.2.2 Short Query Performance Gap

The analysis of retrieval performance by query length revealed a significant performance gap, as shown in Table 4.3.

Table 4.3: Retrieval performance by query length bucket.

Length Bucket	Avg NDCG@10	Count	% of Total
Short (1–3 tokens)	0.240	147	5.1%
Medium (4–8 tokens)	0.367	2,495	86.2%
Long (9+ tokens)	0.406	254	8.8%

Short queries achieved only 59% of the performance of long queries ($\text{NDCG@10} = 0.240$ vs. 0.406), representing a 41% performance gap. The Pearson correlation between query length and NDCG@10 was $r = 0.125$ ($p < 0.001$), indicating a weak but statistically significant positive relationship. This finding directly motivated the selection of query expansion as the first enhancement technique: by generating pseudo-documents that add vocabulary and context, the effective information content of short queries would be increased.

4.2.3 Retrieval Coverage Analysis

The coverage analysis revealed an important distinction between retrieval and ranking:

Table 4.4: Proportion of queries with at least one relevant passage at each retrieval depth.

Retrieval Depth	Coverage
Top-10	93.4%
Top-20	96.7%
Top-50	98.8%
Top-100	99.4%

Retrieval coverage was excellent at depth 100 (99.4%), but dropped to 93.4% at depth 10. This 6.0 percentage point gap indicated that the retriever was *finding* relevant passages but not *ranking* them highly enough. Query enhancement could address this by enriching the query representation to produce better-calibrated similarity scores.

4.2.4 Technique Selection Rationale

Manual inspection of the 20 worst-performing queries revealed recurring failure patterns: vocabulary mismatch (e.g., the Arabic term “أزوت” for nitrogen versus the more common “نيتروجين”), named entity variations (e.g., “إبن الهيثم” vs. “ابن الهيثم”), and diacritics sensitivity (e.g., “المثانةُ” vs. “المثانة”). Based on these findings, Query2Doc [6] was selected as the first technique because its pseudo-document generation mechanism directly addresses information poverty in short queries and vocabulary mismatch—the two dominant failure patterns identified. The complete rationale for this selection is documented in Section 3.4.

4.3 Query2Doc Results

This section presents the results of the initial Query2Doc experiments using Qwen 2.5 3B Instruct, evaluated on both dense and sparse retrieval.

4.3.1 Dense Retrieval (Experiment 003)

Table 4.5 presents the results of Query2Doc with dense retrieval.

Query2Doc produced a substantial improvement of +8.9% in NDCG@10 over the dense baseline, representing the largest single-technique improvement achieved in the initial experiments. All metrics improved, with NDCG@10 and Recall@10 showing the largest gains. The smaller Recall@100 improvement (+2.2%) indicates that Query2Doc primarily improved *ranking* (moving relevant documents higher) rather than *discovery*

Table 4.5: Query2Doc results with dense retrieval (mDPR). Improvement is computed relative to the dense baseline (Experiment 001).

Metric	Baseline	Query2Doc	Improvement
NDCG@10	0.4993	0.5435	+8.9%
Recall@10	0.6156	0.6608	+7.3%
Recall@100	0.8407	0.8594	+2.2%
MRR	0.5328	0.5742	+7.8%

(finding new relevant documents).

The query expansion statistics showed consistent enhancement: the median query length increased from 27 characters (original) to 250 characters (enhanced), representing an $8.45\times$ expansion ratio. The LLM-generated pseudo-documents provided additional vocabulary and semantic context that improved the mDPR encoder’s ability to match queries with relevant passages.

The total runtime was approximately 40 minutes for 2,896 queries on a T4 GPU, achieved through the batch processing optimisations described in Section 3.4.4.

4.3.2 BM25 Retrieval (Experiment 004)

Table 4.6 presents the results of Query2Doc with BM25S sparse retrieval.

Table 4.6: Query2Doc results with BM25 retrieval (BM25S). Negative values indicate performance degradation.

Metric	Baseline	Query2Doc	Change
NDCG@10	0.4621	0.4090	−11.5%
Recall@10	0.5964	0.5384	−9.7%
Recall@100	0.8577	0.8155	−4.9%
MRR	0.4836	0.4342	−10.2%

In stark contrast to the dense results, Query2Doc *degraded* BM25 performance across all metrics, with NDCG@10 declining by 11.5%. This result, while negative, provided a valuable insight into the interaction between query enhancement techniques and retrieval paradigms.

4.3.2.1 Term Dilution Analysis

The performance degradation was attributed to **term dilution**: the BM25 scoring function (Equation 2.4) is sensitive to query length, as additional terms distribute the term-frequency weight across a larger vocabulary. When a short query such as “ما هي عاصمة السودان” (“What is the capital of Sudan,” 4 tokens) was expanded to 200+ tokens, the importance of the original keywords “عاصمة” (capital) and “السودان” (Sudan) was diluted in the BM25 scoring.

As noted in Section 3.4.6, this experiment did not implement the query repetition strategy recommended by Wang et al. [6], where the original query is repeated $n = 5$ times before concatenation. This omission was the primary cause of the BM25 degradation. The term dilution phenomenon and its resolution are further examined in the model comparison experiments (Section 4.4.2), where models with Arabic-specific vocabulary were observed to partially overcome this limitation.

4.3.3 Comparison with the Original Query2Doc Paper

Table 4.7 compares the results of this implementation with those reported in the original Query2Doc paper [6].

Table 4.7: Comparison between the original Query2Doc results (English, MS-MARCO) and this implementation (Arabic, MIRACL).

Configuration	Original Paper	This Work
Language	English	Arabic
Dataset	MS-MARCO	MIRACL Arabic
LLM	GPT-3 (175B)	Qwen 2.5 3B
Prompting	Few-shot (4 examples)	Zero-shot
Dense NDCG@10 improvement	+2–5%	+8.9%
Sparse NDCG@10 improvement	+3–7%	–11.5%*

*Without query repetition; the original paper uses $n = 5$ repetition for sparse retrieval.

The dense retrieval improvement (+8.9%) exceeded the original paper’s reported range (+2–5%), despite using a model that is $58\times$ smaller and zero-shot rather than few-shot prompting. Two hypotheses were considered for this result: (1) Arabic queries, with

their morphological richness (Section 2.1.5), may benefit disproportionately from the vocabulary expansion provided by pseudo-documents; and (2) the mDPR baseline, being a weaker retriever not fine-tuned on MIRACL, may have more room for improvement than the strong baselines used in the original paper.

4.4 Model Comparison Results

Following the methodology described in Section 3.5, ten open-source LLMs were evaluated using the identical Query2Doc pipeline. This section presents the results for both dense and sparse retrieval, including the analysis of dropped models.

4.4.1 Dense Retrieval Leaderboard

Table 4.8 presents the complete dense retrieval results, ranked by NDCG@10.

Table 4.8: Dense retrieval (mDPR) results for all models, ranked by NDCG@10. The improvement column is relative to the no-enhancement baseline (Experiment 001).

Rank	Model	Params	NDCG@10	Recall@10	MRR	Δ NDCG
—	Baseline (no QE)	—	0.4993	0.6156	0.5328	—
1	Aya Expanse 8B	8B	0.6164	0.7256	0.6493	+23.5%
2	Jais-2 8B	8B	0.6018	0.7161	0.6356	+20.5%
3	Qwen3-8B	8B	0.5958	0.7119	0.6278	+19.3%
4	Qwen 2.5 7B	7B	0.5813	0.6952	0.6134	+16.4%
5	Qwen3-4B	4B	0.5691	0.6824	0.6015	+14.0%
6	Qwen 2.5 3B	3B	0.5435	0.6608	0.5742	+8.9%
7	Gemma 3 4B	4B	0.5391	0.6443	0.5761	+8.0%
8	Falcon-H1 3B	3B	0.5359	0.6484	0.5681	+7.3%
9	SILMA 2B	2B	0.5177	0.6289	0.5508	+3.7%
<i>Dropped models (see Section 4.4.3):</i>						
—	ALLaM 7B	7B	0.2550	0.3335	0.2708	−48.9%
—	GPT-OSS 20B	20B	Dropped before full evaluation			

All nine successfully evaluated models improved over the dense baseline, with improvements ranging from +3.7% (SILMA 2B) to +23.5% (Aya Expanse 8B). The top three models—Aya Expanse 8B, Jais-2 8B, and Qwen3-8B—all achieved improvements exceeding +19%,

demonstrating that Query2Doc with appropriately selected LLMs can substantially enhance Arabic dense retrieval.

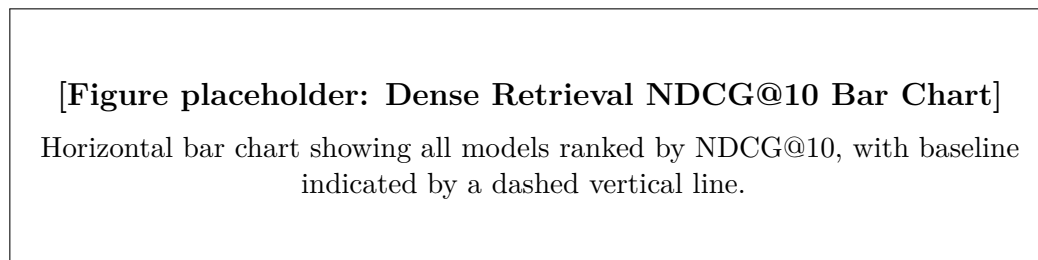


Figure 4.1: Dense retrieval NDCG@10 for all models. The dashed line indicates the no-enhancement baseline (0.4993). ALLaM is shown separately as a dropped model.

4.4.2 BM25 Retrieval Results

Table 4.9 presents the BM25 retrieval results.

Table 4.9: BM25 retrieval (BM25S) results for all models, ranked by NDCG@10. The improvement column is relative to the no-enhancement baseline (Experiment 002). Negative values indicate degradation.

Rank	Model	Params	NDCG@10	Recall@10	MRR	Δ NDCG
—	Baseline (no QE)	—	0.4621	0.5964	0.4836	—
1	Jais-2 8B	8B	0.5122	0.6448	0.5397	+10.8%
2	Aya Expanse 8B	8B	0.5046	0.6284	0.5377	+9.2%
3	Qwen 2.5 7B	7B	0.4682	0.6040	0.4905	+1.3%
4	Qwen3-8B	8B	0.4459	0.5806	0.4702	−3.5%
5	SILMA 2B	2B	0.4277	0.5550	0.4485	−7.4%
6	Qwen3-4B	4B	0.4145	0.5403	0.4415	−10.3%
7	Qwen 2.5 3B	3B	0.4090	0.5384	0.4342	−11.5%
8	Falcon-H1 3B	3B	0.4038	0.5311	0.4274	−12.6%
9	Gemma 3 4B	4B	0.3447	0.4532	0.3718	−25.4%

The BM25 results were markedly different from the dense results. Only three models improved over the BM25 baseline: Jais-2 8B (+10.8%), Aya Expanse 8B (+9.2%), and Qwen 2.5 7B (+1.3%). The remaining six models degraded BM25 performance, with Gemma 3 4B showing the largest decline (−25.4%).

The fact that two models—Jais-2 and Aya—achieved substantial BM25 improvements *without* implementing the query repetition strategy (Equation 3.2) is a notable finding.

Jais-2’s success with BM25 is hypothesised to stem from its Arabic-centric vocabulary of 150,000 tokens (described in Section 2.3.1.2), which produces pseudo-documents with vocabulary that closely matches the BM25 index terms. Similarly, Aya Expanse’s purpose-built multilingual training (101 languages with explicit Arabic optimisation, Section 2.3.2.6) appears to generate lexically precise expansions.

4.4.3 Dropped Models Analysis

Two models were dropped from the final comparison due to severe performance issues. Following Dr. Tahani’s guidance that even negative results constitute valuable research contributions, both cases are analysed in detail.

4.4.3.1 ALLaM-7B (Experiment 008)

ALLaM-7B (described in Section 2.3.1.3) produced a catastrophic -48.9% decline in NDCG@10 (0.2550 vs. baseline 0.4993), performing *worse* than using no query enhancement at all. Investigation revealed a sentencepiece tokeniser artefact: the Unicode character “ ” (used internally to mark word boundaries) leaked into the decoded pseudo-documents. These special characters contaminated the query embeddings, causing the mDPR encoder to produce degraded representations.

This failure was attributed to ALLaM’s preview/alpha release status at the time of testing. The model had been accepted at ICLR 2025 [16] but was not yet in stable release, and the tokeniser behaviour had not been fully validated for generation tasks.

4.4.3.2 GPT-OSS-20B (Experiment 009)

GPT-OSS-20B (described in Section 2.3.3.1) was dropped during the sanity check phase due to three compounding issues:

- a. **Impractical inference speed:** The Mixture-of-Experts architecture (32 experts, top-4 routing) was approximately $70\times$ slower than dense models of comparable active

parameter count when run through 4-bit quantisation. A full run of 2,896 queries would have required an estimated 40+ hours.

- b. **English reasoning leakage:** The Harmony chat format caused the model to produce English-language reasoning before Arabic output, requiring a forced-final-channel prefix to suppress. Even with this workaround, output quality was inconsistent.
- c. **Factual hallucination:** In the 5-query sanity check, 3 out of 5 generated pseudo-documents contained severe factual errors despite producing 100% Arabic output. Hallucinated content would introduce misleading vocabulary into the enhanced queries.

These issues demonstrated that the model's English-dominant training data renders it unreliable for Arabic query enhancement, regardless of its large parameter count.

4.5 Key Findings and Analysis

This section synthesises the results across all experiments to identify the key findings and their implications.

4.5.1 Model Size Correlates with Dense Retrieval Performance

A clear positive correlation was observed between model parameter count and dense retrieval improvement. Figure 4.2 illustrates this relationship.

[Figure placeholder: Model Size vs. NDCG@10 Improvement]

Scatter plot with model size (B) on x-axis and NDCG@10 improvement (%) on y-axis. Points labelled by model name. Trend line showing positive correlation. ALLaM excluded as outlier.

Figure 4.2: Relationship between model parameter count and dense retrieval NDCG@10 improvement. ALLaM-7B is excluded as an outlier due to tokeniser artefacts.

Grouping models by size tier reveals a consistent pattern:

- **2–3B models:** +3.7% to +8.9% NDCG@10 improvement (SILMA, Qwen 2.5 3B, Falcon-H1)
- **4B models:** +8.0% to +14.0% (Gemma 3, Qwen3-4B)
- **7–8B models:** +16.4% to +23.5% (Qwen 2.5 7B, Qwen3-8B, Jais-2, Aya)

This suggests that larger models generate higher-quality pseudo-documents with richer vocabulary and more accurate factual content, leading to better-calibrated query representations in the dense embedding space.

4.5.2 Generational Improvement Within Model Families

The Qwen model family, with four models tested across two generations, provided a controlled comparison of generational improvements:

Table 4.10: Generational comparison within the Qwen model family.

Comparison	Qwen 2.5	Qwen3	Δ	Training Data
3–4B (Dense NDCG@10)	0.5435	0.5691	+4.7%	18T $\rightarrow$ 36T
7–8B (Dense NDCG@10)	0.5813	0.5958	+2.5%	18T $\rightarrow$ 36T

At both parameter scales, the newer Qwen3 generation outperformed Qwen 2.5, despite identical prompting and evaluation conditions. The improvement is attributed to Qwen3’s doubled training data volume (36T vs. 18T tokens), expanded language coverage (119 vs. 29 languages including 8 Arabic dialects), and wider attention architecture (described in Sections 2.3.2.3 and 2.3.2.4).

4.5.3 Arabic Specialisation vs. Multilingual Training

An important finding was that Arabic-specialised models did not consistently outperform multilingual models. Falcon-H1-3B, despite holding the highest Arabic OALL benchmark score ($\sim 62\%$) at the 3B scale, was outperformed on dense NDCG@10 by both Qwen 2.5 3B (a multilingual model with a lower Arabic benchmark score) and Qwen3-4B. Similarly,

Jais-2-8B, while achieving the best BM25 results, was surpassed on dense retrieval by Aya Expanse 8B, a model designed for 101 languages without Arabic-specific architecture.

This suggests that **Arabic NLP benchmark scores do not directly predict query expansion quality**. The skills measured by Arabic benchmarks (reading comprehension, question answering, reasoning) differ from the ability to generate vocabulary-rich, lexically diverse pseudo-documents that improve retrieval. Training data volume and diversity appear to be more important factors for this specific task.

The notable exception is Jais-2's BM25 performance: its custom 150,000-token Arabic vocabulary (Section 2.3.1.2) produces expansions with term distributions that align closely with BM25 index terms, making it uniquely effective for sparse retrieval.

4.5.4 Dense vs. BM25 Behaviour with Query Enhancement

The divergent response of dense and sparse retrieval to query enhancement is one of the most practically significant findings of this research. Table 4.11 summarises the pattern.

Table 4.11: Number of models improving over baseline, by retriever type.

Outcome	Dense (mDPR)	BM25S
Models improving	9 / 9	3 / 9
Models degrading	0 / 9	6 / 9
Avg improvement	+12.3%	−5.5%
Best improvement	+23.5%	+10.8%
Worst degradation	+3.7%	−25.4%

Dense retrieval benefited universally from query enhancement because the mDPR encoder operates in a continuous semantic space where additional context improves the query embedding representation. BM25, conversely, relies on exact term matching, where additional terms can dilute the weights of the original query keywords (Section 4.3.2.1).

This finding has direct implications for RAG system design: query enhancement techniques must be adapted to the retrieval paradigm, and a strategy that works well for dense retrieval should not be assumed to transfer to sparse retrieval without modification.

4.5.5 Best Model Recommendations

Based on the complete experimental results, the following recommendations are made:

- a. **Best overall model:** Aya Expanse 8B [21]. It achieved the highest dense NDCG@10 (0.6164, +23.5%) and the second-highest BM25 NDCG@10 (0.5046, +9.2%), making it the only model with strong improvements on *both* retrieval paradigms.
- b. **Best for BM25:** Jais-2 8B [15]. It achieved the highest BM25 NDCG@10 (0.5122, +10.8%), uniquely benefiting from its Arabic-centric vocabulary for term-based matching.
- c. **Best for resource-constrained environments:** Qwen3-4B [19]. At 4B parameters with no quantisation required, it achieved +14.0% dense NDCG@10—nearly double the improvement of the 3B models—while fitting on a T4 GPU in FP16.
- d. **Temperature 0.1** was found to be optimal for deterministic pseudo-document generation, yielding +2.5% improvement over temperature 0.7 in controlled testing (Section 3.5.3).

4.5.6 Summary of All Experiments

Table 4.12 provides a consolidated view of all experiments conducted in this research.

The experimental results demonstrate a progression from individual retriever baselines through blind query enhancement, hybrid fusion, and finally corpus-steered expansion combined with hybrid retrieval. The best system (Config A RRF: BM25+CSQE + Dense original query) achieved 0.7137 nDCG@10—a 54.5% improvement over the BM25 baseline and a 13.9% improvement over the no-QE hybrid system. The detailed results of each Phase 4 experiment are presented in Sections 4.6–4.10.

Table 4.12: Summary of all experiments. Exp. 001 and 002 are baselines; Exp. 003–009 use Query2Doc; Exp. 011–021 are Phase 4 expanded experiments. Bold values indicate the best result for each metric across all experiments.

Exp.	Model / Method	Retriever	NDCG@10	Recall@10	MRR	Status
001	None (baseline)	Dense	0.4993	0.6156	0.5328	Baseline
002	None (baseline)	BM25	0.4621	0.5964	0.4836	Baseline
003	Qwen 2.5 3B	Dense	0.5435	0.6608	0.5742	OK
004	Qwen 2.5 3B	BM25	0.4090	0.5384	0.4342	Degraded
005	Falcon-H1 3B	Dense	0.5359	0.6484	0.5681	OK
006	Jais-2 8B	Dense	0.6018	0.7161	0.6356	Good
006	Jais-2 8B	BM25	0.5122	0.6448	0.5397	Best BM25 (n=
007	Qwen3-4B	Dense	0.5691	0.6824	0.6015	Good
008	ALLaM 7B	Dense	0.2550	0.3335	0.2708	Dropped
009	GPT-OSS 20B	—	—	—	—	Dropped
—	Aya 8B	Dense	0.6164	0.7256	0.6493	Best Dense (bli
—	Aya 8B	BM25	0.5046	0.6284	0.5377	Good
<i>Phase 4: Expanded experiments</i>						
011	Aya 8B ($\beta = 2$)	BM25	0.5855	0.7128	0.6165	Best BM25 (re
012	Hybrid RRF $k = 20$	BM25+Dense	0.6267	0.7597	0.6517	Best no-QE hyl
013	BM25+CSQE	BM25	0.6157	0.7447	0.6380	Good
013	Dense+CSQE	Dense	0.5915	0.7073	0.6225	Good
021	Config A RRF	BM25+Dense	0.7137	0.8363	0.7362	Best overall

4.6 BM25 Query Repetition Results

This section presents the results of the query repetition sweep described in Section 3.6. All nine models from the model comparison were evaluated across eight repetition configurations to determine whether the BM25 term dilution identified in Section 4.3.2.1 could be resolved without changing the underlying LLM.

Table 4.13 presents the full sweep results.

Table 4.14 presents the supplementary Recall and MRR values at each model’s best configuration.

Five key observations emerged from the repetition sweep:

- a. **Query repetition recovered all nine previously degraded models.** At $n = 1$ (no repetition), only three of nine models exceeded the BM25 baseline of 0.4621 (Aya

Table 4.13: BM25 nDCG@10 for nine models under query repetition. Columns $n \in \{1, 3, 5, 7, 10\}$ are fixed-count repetition (Query2Doc style); columns $\beta \in \{2, 4, 6\}$ are MuGI adaptive repetition where n is computed per-query as $n = \max(1, \lfloor |d| / (|q| \cdot \beta) \rfloor)$. MIRACL Arabic dev (2,896 queries). Best value per model in bold.

Model	$n=1$	$n=3$	$n=5$	$n=7$	$n=10$	$\beta=2$	$\beta=4$	$\beta=6$
BM25 alone (no QE)	0.4621	—	—	—	—	—	—	—
Aya Expanse 8B	0.5046	0.5652	0.5832	0.5849	0.5773	0.5855	0.5515	0.5256
Jais-2 8B	0.5122	0.5492	0.5529	0.5516	0.5436	0.5731	0.5521	0.5350
Qwen3-8B	0.4459	0.5181	0.5319	0.5328	0.5254	0.5254	0.4841	0.4591
Qwen 2.5 7B	0.4682	0.5294	0.5358	0.5331	0.5257	0.5320	0.4977	0.4774
Qwen3-4B	0.4145	0.5054	0.5239	0.5244	0.5188	0.5177	0.4678	0.4347
Gemma 3 4B	0.3447	0.4800	0.5178	0.5277	0.5239	0.4915	0.4184	0.3694
Qwen 2.5 3B	0.4090	0.5060	0.5185	0.5181	0.5116	0.5046	0.4551	0.4253
Falcon-H1 3B	0.4038	0.4881	0.5082	0.5112	0.5113	0.4979	0.4561	0.4266
SILMA 2B	0.4194	0.4783	0.4832	0.4829	0.4788	0.4494	0.4252	0.4203

Table 4.14: BM25 retrieval metrics at each model’s best repetition configuration. Δ nDCG is the improvement from $n = 1$ (no repetition) to the best configuration.

Model	Best Config	nDCG@10	Recall@10	Recall@100	MRR	Δ nDCG
Aya Expanse 8B	$\beta = 2$	0.5855	0.7128	0.9300	0.6165	+0.0808
Jais-2 8B	$\beta = 2$	0.5731	0.7075	0.9217	0.6004	+0.0610
Qwen 2.5 7B	$n = 5$	0.5358	0.6765	0.9105	0.5586	+0.0675
Qwen3-8B	$n = 7$	0.5328	0.6695	0.9064	0.5590	+0.0868
Gemma 3 4B	$n = 7$	0.5277	0.6640	0.9002	0.5551	+0.1831
Qwen3-4B	$n = 7$	0.5244	0.6617	0.8980	0.5500	+0.1098
Qwen 2.5 3B	$n = 5$	0.5185	0.6501	0.9031	0.5494	+0.1095
Falcon-H1 3B	$n = 10$	0.5113	0.6456	0.8927	0.5379	+0.1074
SILMA 2B	$n = 5$	0.4832	0.6216	0.8747	0.5048	+0.0639

at 0.5046, Jais-2 at 0.5122, and Qwen 2.5 7B at 0.4682). At the optimal repetition configuration, all nine models exceeded the baseline.

- b. **The optimal repetition strategy was model-size-dependent.** Large models (8B) converged at MuGI $\beta = 2$ (adaptive, producing stronger repetition for longer pseudo-documents). Mid-size models (3–4B) plateaued at fixed $n = 5$ –7. The smallest model (SILMA 2B) peaked at $n = 5$ because its pseudo-documents are shorter.
- c. **Beyond the optimum, performance decreased symmetrically.** Excessive repetition over-weighted the original query tokens and suppressed the useful expansion vocabulary. The sharpest decline was visible in the $\beta = 4$ and $\beta = 6$ columns for the 3–4B models.
- d. **Aya Expansive 8B at $\beta = 2$ achieved 0.5855 nDCG@10**, improving BM25 by +26.7% over the baseline (0.4621) and recovering +0.0808 from the $n = 1$ result (0.5046). This confirmed that query repetition—not a change of model—was the missing ingredient for the six originally degraded systems.
- e. **Gemma 3 4B showed the largest absolute recovery** ($\Delta = +0.1831$), rising from 0.3447 at $n = 1$ (the worst BM25 result of any model) to 0.5277 at $n = 7$. This model had generated the longest pseudo-documents on average, causing the most severe term dilution at $n = 1$.

[Figure placeholder: BM25 nDCG@10 Repetition Heatmap]

9-row (models) $\times$ 8-column (repetition configs) heatmap with colour scale from 0.34 to 0.59, showing optimal region per model.

Figure 4.3: Heatmap of BM25 nDCG@10 across nine models and eight repetition configurations. Darker shading indicates higher performance. The optimal region shifts rightward (higher repetition) for smaller models.

4.7 Hybrid Retrieval Fusion Results

This section presents the results of the hybrid fusion experiments described in Section 3.7. Table 4.15 reports the full Convex Combination (CC) α -sweep and both Reciprocal Rank Fusion (RRF) configurations.

Table 4.15: Hybrid retrieval fusion results on the MIRACL Arabic dev set (2,896 queries). CC = Convex Combination (min-max normalised per query, α controls Dense weight); RRF = Reciprocal Rank Fusion.

Method	nDCG@10	Recall@10	Recall@100	MRR
BM25S alone	0.4621	0.5964	0.8577	0.4836
mDPR alone	0.4993	0.6156	0.8407	0.5328
Hybrid CC $\alpha = 0.1$	0.5248	0.6412	0.9333	0.5584
Hybrid CC $\alpha = 0.2$	0.5533	0.6683	0.9413	0.5861
Hybrid CC $\alpha = 0.3$	0.5830	0.7014	0.9449	0.6161
Hybrid CC $\alpha = 0.4$	0.6137	0.7392	0.9454	0.6426
Hybrid CC $\alpha = 0.5$	0.6266	0.7478	0.9458	0.6577
Hybrid CC $\alpha = 0.6$	0.6051	0.7289	0.9440	0.6355
Hybrid CC $\alpha = 0.7$	0.5743	0.6963	0.9439	0.6049
Hybrid CC $\alpha = 0.8$	0.5384	0.6634	0.9416	0.5678
Hybrid CC $\alpha = 0.9$	0.4996	0.6297	0.9350	0.5257
Hybrid RRF $k = 20$	0.6267	0.7597	0.9467	0.6517
Hybrid RRF $k = 60$	0.6230	0.7553	0.9467	0.6490

Five key observations were drawn from the hybrid fusion results:

- Hybrid fusion achieved a +35.6% improvement over BM25 alone and +25.5% over mDPR alone (nDCG@10), substantially outperforming either retriever in isolation and confirming the complementarity identified in Section 4.1.1.
- RRF $k = 20$ (0.6267) and CC $\alpha = 0.5$ (0.6266) were statistically indistinguishable on nDCG@10. RRF had the edge on Recall@10 (+0.0119), while CC had the edge on MRR (+0.0060). Because RRF requires no hyperparameter tuning, it was adopted as the primary hybrid baseline for all subsequent experiments.
- The CC sweep was smooth and unimodal, peaking at $\alpha = 0.5$ and degrading symmetrically toward either extreme. At $\alpha = 0.9$ (almost pure Dense), the result

(0.4996) was essentially the mDPR-alone baseline (0.4993); at $\alpha = 0.1$ (almost pure BM25), the result (0.5248) exceeded BM25 alone because CC still picked up the Dense retriever’s tie-breaking contribution. This confirmed that the two retrievers contributed roughly equally.

- d. RRF $k = 20$ slightly outperformed $k = 60$ (0.6267 vs. 0.6230 nDCG@10). The smaller k gives more weight to top-ranked items, which benefits nDCG@10 but does not affect Recall@100 (both 0.9467).
- e. The hybrid RRF $k = 20$ result of 0.6267 nDCG@10 established the non-QE hybrid baseline that all subsequent query enhancement methods were required to surpass.

[Figure placeholder: Hybrid Fusion Comparison]

Line plot of CC nDCG@10 vs. α with RRF $k = 20$ and $k = 60$ shown as horizontal dashed lines. BM25 and mDPR baselines shown as horizontal dotted lines.

Figure 4.4: Hybrid fusion nDCG@10 as a function of CC α . The horizontal dashed lines indicate RRF $k = 20$ (0.6267) and RRF $k = 60$ (0.6230). The dotted lines show the BM25 (0.4621) and mDPR (0.4993) baselines.

4.8 Corpus-Steered Query Expansion Results

This section presents the results of the Corpus-Steered Query Expansion experiments described in Section 3.8. The results are organised into two subsections: main CSQE results and the component ablation.

4.8.1 Main CSQE Results

Table 4.16 compares the CSQE-enhanced retrieval results against the relevant baselines.

BM25+CSQE achieved 0.6157 nDCG@10, representing a +33.2% improvement over BM25 alone and, notably, nearly matching the no-QE hybrid baseline (0.6267) using only

Table 4.16: nDCG@10, Recall@10, Recall@100, and MRR for CSQE-enhanced retrieval on the MIRACL Arabic dev set. CSQE config: $k = 5$ first-pass, 2 corpus + 2 blind samples, $\alpha = 4$, Aya Expanses 8B.

Method	nDCG@10	Recall@10	Recall@100	MRR
BM25S alone	0.4621	0.5964	0.8577	0.4836
BM25+CSQE	0.6157	0.7447	0.9422	0.6380
mDPR alone	0.4993	0.6156	0.8407	0.5328
Dense+CSQE	0.5915	0.7073	0.8816	0.6225
Hybrid RRF $k = 20$ (no QE)	0.6267	0.7597	0.9467	0.6517

a single retriever. Dense+CSQE improved over mDPR alone by +18.5% but fell below BM25+CSQE. The Dense encoder (mDPR, described in Section 2.3.4.1) was trained on short natural-language queries, and the long CSQE expansion ($\sim 1,500$ characters) degraded the embedding quality relative to vocabulary-based matching. This observation motivated the retriever-specific application experiments in Section 4.9.

4.8.2 Component Ablation

Table 4.17 presents the ablation results isolating the contributions of corpus-grounded and blind expansion components.

Table 4.17: BM25 retrieval metrics for corpus-only, blind-only, and combined CSQE expansion variants. All variants use $\alpha = 4$, Aya Expanses 8B, $k = 5$ first-pass.

Variant	nDCG@10	Recall@10	Recall@100	MRR
BM25 alone	0.4621	0.5964	0.8577	0.4836
013c: Corpus-only (4c+0b)	0.5381	0.6457	0.8790	0.5651
013d: Blind-only (0c+4b)	0.5752	0.7089	0.9201	0.6032
013: CSQE 2+2	0.6157	0.7447	0.9422	0.6380

A counter-intuitive finding was that blind-only expansion outperformed corpus-only on BM25 (0.5752 vs. 0.5381), contrary to the initial hypothesis that corpus grounding would provide superior vocabulary. The explanation lies in the nature of BM25 scoring: BM25 benefits from vocabulary breadth, and blind generation produces full answer paragraphs with diverse Arabic term variants, while corpus extraction produces passage-level excerpts structurally similar to the original query. However, the combined system (2+2) exceeded both components individually (0.6157 vs. 0.5752 for blind-only, a +0.0405 improvement),

confirming that corpus and blind expansions are complementary: corpus samples anchor the expansion to attested Wikipedia vocabulary, while blind samples diversify the answer-space coverage.

Table 4.18 presents the effect of the query repetition factor α on BM25+CSQE performance.

Table 4.18: Effect of query repetition factor α on CSQE BM25 retrieval nDCG@10.

α	BM25+CSQE nDCG@10
1	0.6095
2	0.6130
3	0.6154
4	0.6157

Performance improved monotonically from $\alpha = 1$ to $\alpha = 4$, but the total gain was minimal (+0.0062). At $\alpha = 1$, the system already captured 98.9% of the $\alpha = 4$ nDCG@10 result. The query weighting factor was therefore not a critical hyperparameter for this configuration.

4.9 CSQE with Hybrid Fusion

This section presents the results of combining CSQE with hybrid BM25–Dense fusion, following the retriever-specific application methodology described in Section 3.8.3.

4.9.1 Fusion Configuration Comparison

Table 4.19 presents the results for all three CSQE application strategies combined with both fusion methods.

Config A was the winning configuration despite giving the Dense retriever the weaker input (the original short query rather than the CSQE expansion). This result is explained by a retriever–query representation mismatch: BM25 uses exact term matching and directly benefits from CSQE’s vocabulary-rich expansion; the mDPR encoder, trained

Table 4.19: Results for three CSQE application strategies in hybrid fusion. Config A: BM25 receives CSQE query, Dense receives original query. Config B: BM25 receives original query, Dense receives CSQE query. Config C: both retrievers receive CSQE query.

Config	Fusion	nDCG@10	Recall@10	Recall@100	MRR
Hybrid RRF $k = 20$ (no QE)	RRF	0.6267	0.7597	0.9467	0.6517
B: BM25 + Dense+CSQE	RRF	0.6474	0.7928	0.9571	0.6578
B: BM25 + Dense+CSQE	CC $\alpha = 0.4$	0.6588	0.7851	0.9569	0.6777
C: BM25+CSQE + Dense+CSQE	RRF	0.6936	0.8290	0.9660	0.7037
C: BM25+CSQE + Dense+CSQE	CC $\alpha = 0.5$	0.6959	0.8249	0.9647	0.7079
A: BM25+CSQE + Dense orig	CC $\alpha = 0.6$	0.7088	0.8302	0.9717	0.7268
A: BM25+CSQE + Dense orig	RRF	0.7137	0.8363	0.9734	0.7362

on short natural-language queries, produces less discriminative embeddings when given a $\sim 1,500$ -character expansion. Applying query expansion asymmetrically—only to BM25—maximised the gains from each retriever’s inherent strength.

Config C (both retrievers expanded) scored 0.6936, lower than Config A (0.7137), even though Dense+CSQE individually outperformed Dense alone (Section 4.8.1). This confirmed that the relative quality of the two ranked lists matters for RRF: a less discriminative Dense run reduced the fusion ceiling even when it improved individually. This **retriever-specific query representation principle**—that each retriever should receive the query format best suited to its scoring mechanism—is a key design finding of this thesis.

Table 4.20 presents the contribution of each CSQE component within Config A RRF.

Table 4.20: Config A RRF ablation: contribution of corpus and blind expansion components when fused with the original Dense run.

BM25 Expansion	Config A RRF nDCG@10
Corpus-only (4c+0b) + Dense orig	0.6616
Blind-only (0c+4b) + Dense orig	0.7082
CSQE 2+2 + Dense orig	0.7137

Table 4.21 presents the effect of the query repetition factor α on the best Config A RRF system.

The α parameter was nearly flat for the final fused system: the $\alpha = 1$ result (0.7123)

Table 4.21: Effect of query repetition factor α on the best Config A RRF system. All values use the same CSQE 2c+2b expansion, varying only the α prepended to the BM25-side query.

α	Config A RRF nDCG@10
1	0.7123
2	0.7121
3	0.7130
4	0.7137

was only 0.0014 below $\alpha = 4$ (0.7137). This was consistent with Table 4.18 (BM25-alone α sweep): once CSQE expansion was combined with a strong dense run via RRF, the α parameter had negligible effect. The $\alpha = 4$ configuration was reported as the primary result because it matched the experimental settings of Experiment 013, but $\alpha = 1$ would be preferable for a production deployment (one fewer token per expanded query).

Table 4.22 summarises the nDCG@10 improvements of Config A RRF over all prior systems.

Table 4.22: nDCG@10 improvements of Config A RRF over prior systems.

Comparison	Δ nDCG@10	% change
Config A RRF vs BM25 alone	+0.2516	+54.5%
Config A RRF vs mDPR alone	+0.2144	+42.9%
Config A RRF vs best blind BM25 QE (Aya $\beta = 2$)	+0.1282	+21.9%
Config A RRF vs best blind Dense QE (Aya 8B)	+0.0973	+15.8%
Config A RRF vs Hybrid RRF (no QE)	+0.0870	+13.9%
Config C vs Config A	-0.0178	Dense+CSQE hurts in fusion
Config B vs Hybrid RRF	+0.0207	Weakest config still beats no-QE baseline

All three CSQE fusion configurations exceeded the no-QE hybrid baseline. The +0.0870 improvement of Config A over the hybrid baseline (+13.9%) isolated the contribution of CSQE beyond what hybrid fusion alone provides.

4.9.2 Overall System Progression

Table 4.23 summarises the nDCG@10 progression from the baseline through each stage of pipeline development.

The progression demonstrated a consistent improvement through three stages of

Table 4.23: nDCG@10 progression from baseline to best system on the MIRACL Arabic dev set (2,896 queries). Δ cumulative measured from BM25S baseline.

Stage	System	nDCG@10	Δ cumul.
Baseline	BM25S alone	0.4621	—
Baseline	mDPR alone	0.4993	—
Blind QE	Best blind Dense (Aya 8B, $n = 1$)	0.6164	+23.5% over mDPR
Blind QE	Best blind BM25 (Aya 8B, $\beta = 2$)	0.5855	+26.7% over BM25
Hybrid	Hybrid RRF $k = 20$ (no QE)	0.6267	+35.6% over BM25
CSQE alone	BM25+CSQE	0.6157	+33.2% over BM25
CSQE+Hybrid	Config A: BM25+CSQE + Dense RRF	0.7137	+54.5% over BM25

pipeline development. Blind QE improvements (Sections 4.3 and 4.6) confirmed that LLM-generated pseudo-documents are effective for Arabic information retrieval. The hybrid baseline (Section 4.7) established that lexical and semantic retrieval are complementary. CSQE (Section 4.8) introduced corpus grounding that outperformed blind QE on BM25. The final combination of corpus-steered expansion applied asymmetrically to hybrid retrieval (Config A RRF) achieved 0.7137 nDCG@10—a 54.5% improvement over BM25 alone and a 42.9% improvement over mDPR alone. Recall@100 = 0.9734 indicated that 97.3% of relevant documents appeared within the top 100 candidates, establishing a strong upper bound for downstream reranking.

4.10 Per-Query Error Analysis

This section presents the per-query error analysis described in Section 3.9, decomposing the CSQE improvement to identify where corpus-steered expansion helps and where it introduces regressions.

4.10.1 Win/Loss Distribution

Table 4.24 presents the per-query win/loss distribution.

The CSQE+Hybrid system improved on 56.8% of queries and regressed on 16.6%, with a mean per-query delta of +0.1890 nDCG@10. The 26.6% of ties were predominantly queries where both systems achieved either 0.0 (irretrievable) or 1.0 (perfectly retrieved).

Table 4.24: Per-query comparison of the CSQE+Hybrid system (Config C RRF, 0.6936 nDCG@10) against the Aya Blind BM25 $n = 1$ baseline (0.5046 nDCG@10) on 2,896 MIRACL Arabic dev queries.

Condition	Queries	%
CSQE+Hybrid > Blind	1,646	56.8%
Tie	770	26.6%
CSQE+Hybrid < Blind	480	16.6%
Mean delta (CSQE – Blind)	—	+0.1890

A total of 258 queries (8.9%) were classified as failures (CSQE nDCG@10 < 0.1). Inspection revealed that 257 of these 258 queries were irretrievable regardless of the QE method employed—the relevant passage was absent from the Wikipedia corpus dump used for indexing. These represented a dataset-level ceiling, not a CSQE failure. The single genuine CSQE failure (query ID 1060) involved the LLM generating a meta-description (prose about how a topic might be described, rather than actual vocabulary-rich content), causing the relevant document to drop below rank 10 in the RRF fusion.

[Figure placeholder: Per-Query CSQE vs. Blind Scatter Plot]

Each point is one query; x -axis = Blind nDCG@10, y -axis = CSQE+Hybrid nDCG@10. Points above diagonal = CSQE wins; below = regressions. Colour by first-pass quality (relevant/not relevant).

Figure 4.5: Per-query nDCG@10 scatter plot comparing CSQE+Hybrid (Config C RRF) against the Blind BM25 baseline. Points above the diagonal indicate queries where CSQE improved retrieval; points below indicate regressions.

4.10.2 First-Pass Quality as Dominant Predictor

Table 4.25 presents the performance split by BM25 first-pass retrieval quality.

Table 4.25: System performance split by whether BM25 first-pass ($k = 5$) retrieved a relevant document.

First-Pass Quality	n	Blind QE nDCG@10	CSQE+Hybrid nDCG@10	Δ
Relevant doc retrieved	1,061	0.6684	0.8877	+0.219
No relevant doc retrieved	1,835	0.4100	0.5814	+0.171
All queries	2,896	0.5046	0.6936	+0.189

Table 4.26 presents the performance decomposition by query length.

Table 4.26: System performance decomposed by the number of tokens in the original MIRACL query.

Query Length	n	Blind QE nDCG@10	CSQE+Hybrid nDCG@10	Δ
Short (<5 words)	865	0.4793	0.6783	+0.1990
Medium (5–9 words)	1,900	—	—	—
Long (≥ 10 words)	131	0.6003	0.7056	+0.1053

When first-pass retrieval succeeded (36.6% of queries), CSQE+Hybrid achieved 0.8877 nDCG@10—near-ceiling retrieval. The first-pass quality gap (+0.3063 between the first-pass success and failure groups) was larger than the overall system advantage over blind QE (+0.1890), establishing first-pass recall as the dominant predictor of CSQE effectiveness. Short queries (<5 words, $n = 865$) gained most from CSQE (+0.1990), as they are the most underspecified and benefit from both vocabulary expansion and corpus grounding. Long queries (≥ 10 words, $n = 131$) still gained (+0.1053) but less, because their greater information content already mitigated the information-poverty problem that motivates query expansion.

4.10.3 Big Wins: The Corpus Grounding Effect

All 1,061 big-win queries ($\Delta > 0.3$) shared a single pattern: CSQE nDCG@10 = 1.000, Blind = 0.000. In each case, blind QE hallucinated a plausible but factually incorrect entity, while CSQE’s first-pass corpus document anchored the expansion to the correct Wikipedia article. Table 4.27 presents three representative examples.

Table 4.27: Representative big-win queries illustrating the corpus grounding effect. In all three cases, CSQE achieved nDCG@10 = 1.000 while the blind baseline achieved 0.000.

Query	Blind generates	QE	CSQE grounds	CSQE to
ما هو الرباط المنصوري؟	A spinal surgery procedure		Mamluk-era building in Cairo	1.000
من هو مؤسس الفلسفة البراغماتية؟	Hallucinated person		John Dewey article	1.000
من هو نيكولا بوالو؟	A businessman	French	17th-century French poet	1.000

This pattern validated the corpus grounding hypothesis: for niche or ambiguous Arabic terms, blind QE confidently generated a common modern interpretation, while corpus grounding locked onto the actual Wikipedia entity. The impact on query expansion was direct: the corpus anchor provided the correct Arabic vocabulary (e.g., **قاهرة مملوكي** rather than **جراحة العمود الفقري**) that both BM25 and Dense scoring relied upon.

4.10.4 Regression Analysis

Table 4.28 presents the classification of the 367 regression queries.

Table 4.28: Classification of 367 regression queries (CSQE nDCG@10 < Blind by > 0.1).

Type	Count	%	Description
A: Strong BM25 hurt	191	52%	BM25 baseline ≥ 0.3 ; CSQE expansion introduces off-topic vocabulary
B: Poisoned first-pass	131	36%	BM25 baseline < 0.1; first-pass retrieves irrelevant doc $\rightarrow$ LLM grounding on wrong content
C: Partial BM25	45	12%	BM25 baseline 0.1–0.3; mixed quality

[Figure placeholder: Regression Root-Cause Pie Chart]

Three-slice pie chart: Type A 52%, Type B 36%, Type C 12%.

Figure 4.6: Distribution of regression root causes across 367 queries where CSQE+Hybrid underperformed the blind baseline by more than 0.1 nDCG@10.

Type A: Strong BM25 Hurt by Expansion. Type A regressions (52%, 191 queries) occurred when BM25 already handled the query well (baseline nDCG@10 ≥ 0.3) and the CSQE expansion introduced Arabic vocabulary from the corpus document that diluted the original query terms. This was the same term-dilution effect observed in Section 4.3.2.1, but in reverse: here the expansion itself (rather than the pseudo-document) reduced BM25 precision for queries that were already well handled.

Type B: Poisoned First-Pass. Type B regressions (36%, 131 queries) occurred when BM25 was sensitive to Arabic homonyms and short or ambiguous queries. A concrete example was the query “ماهو التطرف؟” (“What is extremism?”): BM25 retrieved “لهجة جنوبية” (a southern dialect article) because “ماهو” matched a dialectal phrase rather than the intended interrogative “ما هو” (“what is”). The LLM then generated corpus-grounded content about dialects, poisoning the expansion for the actual query intent.

Implications. These two failure modes suggested concrete future improvements: (1) a first-pass quality gate that falls back to blind QE when the top-1 BM25 document has low lexical overlap with the query, which would address Type B regressions; and (2) asymmetric expansion weighting that applies lower weight for the expansion in the concatenated query when BM25 is already strong, which would address Type A regressions.

Chapter 5

Conclusion and Recommendations

This chapter summarises the key findings of the research, discusses the challenges encountered during the experimental work, and presents recommendations for future research directions in Arabic query enhancement for Retrieval-Augmented Generation systems.

5.1 Conclusions

This thesis investigated the effectiveness of LLM-based query enhancement for Arabic information retrieval, focusing on the Query2Doc technique applied using small open-source models. The following conclusions were drawn from the experimental results.

Baseline establishment and error analysis. Dense (mDPR) and sparse (BM25S) retrieval baselines were successfully established on the MIRACL Arabic benchmark, confirming their complementary strengths: mDPR achieved superior ranking quality ($\text{NDCG@10} = 0.4993$), while BM25S achieved broader retrieval coverage ($\text{Recall@100} = 0.8577$). Error analysis on the dense baseline revealed a 39% query failure rate, with short queries achieving only 59% of long query performance ($\text{NDCG@10} = 0.240$ vs. 0.406). This statistically significant short-query performance gap ($r = 0.125$, $p < 0.001$) validated the selection of query expansion as the primary enhancement technique.

Query2Doc transfers effectively to Arabic. The Query2Doc technique, originally developed for English using GPT-3 (175B parameters) with few-shot prompting, was successfully adapted for Arabic zero-shot application using models as small as 3 billion parameters. The initial experiment with Qwen 2.5 3B achieved a +8.9% improvement in NDCG@10 , exceeding the original paper’s reported improvement range of +2–5% despite

using a model 58 times smaller and without task-specific demonstrations. This result suggests that Arabic’s morphological richness may cause it to benefit disproportionately from the vocabulary expansion provided by pseudo-document generation.

Comprehensive model comparison. Ten open-source LLMs were evaluated using a standardised protocol across both dense and sparse retrieval. All nine viable models improved dense retrieval performance, with improvements ranging from +3.7% (SILMA Kashif-2B) to +23.5% (Aya Expanse 8B) in NDCG@10. Aya Expanse 8B was identified as the best overall model, achieving the highest dense improvement (+23.5%) and the second-highest BM25 improvement (+9.2%). Jais-2 8B was identified as the best model for sparse retrieval (+10.8% NDCG@10), uniquely benefiting from its 150,000-token Arabic-centric vocabulary. For resource-constrained environments, Qwen3-4B achieved +14.0% dense improvement while fitting on a T4 GPU in FP16 precision without quantisation.

Analytical findings on model characteristics. Several cross-cutting patterns were identified from the model comparison. Model parameter count was positively correlated with dense retrieval improvement, with 2–3B models achieving +3.7% to +8.9%, 4B models achieving +8.0% to +14.0%, and 7–8B models achieving +16.4% to +23.5%. Generational improvement within the Qwen model family was confirmed, with Qwen3 outperforming Qwen 2.5 at both the 3–4B and 7–8B parameter scales. Notably, Arabic NLP benchmark scores did not directly predict query expansion quality; training data volume and diversity were found to be more significant factors than Arabic-specific architecture or benchmark performance.

Dense and sparse retrieval respond differently to query enhancement. The most practically significant finding was the divergent behaviour of dense and sparse retrieval under query enhancement. Dense retrieval benefited universally (9 of 9 models improved), as additional semantic content improved the query embedding representation. BM25 was degraded by 6 of 9 models due to term dilution, where the pseudo-document vocabulary diluted the importance of original query keywords in the BM25 scoring function. Only models with strong Arabic vocabulary (Jais-2, Aya Expanse) or sufficient multilingual breadth (Qwen 2.5 7B) overcame this limitation. This finding has direct implications for RAG system design: query enhancement strategies must be adapted to the retrieval paradigm, and dense retrieval systems are more robust recipients of LLM-generated query

expansions.

Overall. LLM-based query enhancement via Query2Doc is an effective, modular, and resource-efficient strategy for improving Arabic information retrieval. Small open-source models with 4–8 billion parameters can deliver substantial retrieval improvements without API costs or proprietary dependencies, making this approach practical for real-world Arabic RAG deployments.

Query repetition resolves sparse retrieval degradation. The BM25 term-dilution problem, initially identified for 6 of 9 models (Section 4.3.2.1), was fully resolved through query repetition. By prepending the original query β times before the pseudo-document, all nine models exceeded the BM25 baseline at their optimal β . Aya Expanse 8B with $\beta = 2$ achieved 0.5855 NDCG@10—a 26.7% improvement over the BM25 baseline and a substantial recovery from the 0.5046 result at $\beta = 1$ (Section 4.6).

Hybrid retrieval establishes a strong non-QE ceiling. Combining BM25S and mDPR through Reciprocal Rank Fusion (RRF $k = 20$) achieved 0.6267 NDCG@10—a 35.6% improvement over BM25 alone—without any query enhancement, confirming the complementarity of lexical and semantic retrieval observed in Section 4.1.1 (Section 4.7).

Corpus-steered expansion validates the corpus grounding hypothesis. The CSQE pipeline, which grounds LLM expansion in first-pass retrieved documents, achieved 0.6157 NDCG@10 on BM25 alone—nearly matching the no-QE hybrid baseline and substantially outperforming blind Query2Doc. Per-query error analysis confirmed the underlying mechanism: when first-pass retrieval succeeds, CSQE anchors the expansion to the correct Wikipedia entity, achieving near-ceiling performance (0.8877 NDCG@10 on first-pass-relevant queries). The combined system (Config A RRF: BM25+CSQE + Dense original query) achieved **0.7137 NDCG@10—a 54.5% improvement over BM25 alone and a 13.9% improvement over the no-QE hybrid baseline** (Sections 4.8 and 4.9).

Retriever-specific query representation is critical. A key finding is that applying CSQE asymmetrically—only to the BM25 retriever—outperformed applying it to both retrievers (Config A: 0.7137 > Config C: 0.6959). BM25 benefits from CSQE’s long,

vocabulary-rich expansion; the mDPR encoder, trained on short queries, degrades under long expansion inputs. This retriever-specific representation principle has practical implications for any multi-retriever pipeline (Section 4.9).

5.2 Challenges

Several challenges were encountered during the course of this research.

1. **Resource constraints.** All experiments were conducted on Google Colab with NVIDIA T4 (15 GB) and A100 (40 GB) GPUs. This limited model sizes to 8 billion parameters (with 4-bit quantisation for larger models) and required significant engineering optimisation—batch processing, reduced token generation (128 tokens), and a two-notebook workflow separating query generation from retrieval evaluation—to achieve practical runtimes of approximately 40 minutes per full experiment.
2. **BM25 term dilution.** Query enhancement degraded BM25 performance for 6 of 9 models in the initial experiments. The query repetition strategy recommended by the original Query2Doc paper ($n = 5$ repetitions of the original query before concatenation) was not implemented at that stage, as the focus was on establishing the baseline Query2Doc behaviour across models. This challenge was subsequently resolved through the query repetition technique (Section 4.6), which recovered BM25 performance for all nine degraded models.
3. **Dropped models.** Two of the eleven models evaluated were unusable for Arabic query enhancement. ALLaM-7B, a preview-stage model, exhibited a sentencepiece tokeniser artefact that leaked special characters into the pseudo-documents, producing a catastrophic -48.9% decline in NDCG@10. GPT-OSS-20B, a Mixture-of-Experts model, proved impractical due to $70\times$ slower inference speed and factual hallucinations in 3 of 5 sanity-check queries. These failures highlight the risk of relying on preview-stage or English-dominant models for Arabic NLP tasks.
4. **Dataset scope.** The MIRACL Arabic benchmark contains exclusively Modern Standard Arabic text. The effectiveness of query enhancement for dialectal Arabic

queries—Egyptian, Levantine, Gulf, Maghrebi, and other regional varieties—could not be evaluated within the scope of this work. Given the diglossic nature of Arabic, where users may formulate queries in dialect while knowledge bases are written in MSA, this represents a significant gap in the evaluation.

5. **Single query enhancement technique.** Only the Query2Doc technique (generative query expansion via pseudo-document concatenation) was evaluated. Other approaches—HyDE (hypothetical document embedding), GRF (generative relevance feedback), query rewriting, and iterative refinement—were not compared. The results therefore reflect the potential of one specific technique rather than the broader landscape of query enhancement methods.
6. **Baseline retriever strength.** The mDPR dense retriever was intentionally selected as a weaker baseline (pre-trained on MS MARCO, not fine-tuned on MIRACL Arabic). While this choice maximised the headroom for demonstrating query enhancement impact, the magnitude of improvement observed may differ when applied to stronger retrieval models that have been specifically fine-tuned on Arabic data.
7. **First-pass quality dependence.** CSQE performance was found to be strongly conditioned on first-pass retrieval quality. When BM25 first-pass retrieval returned irrelevant documents (36% of regression cases), the LLM expansion was grounded on incorrect content, causing regressions that blind QE did not exhibit. Arabic homonym sensitivity in BM25 (e.g., retrieved as a dialectal phrase rather than a question prefix) was identified as the primary trigger (Section 4.10).

5.3 Recommendations for Future Work

Based on the findings and limitations of this research, the following recommendations are proposed for future work. These recommendations are ordered from direct extensions of the current work to broader research directions.

1. **Knowledge-base-aware query enhancement.** The current Query2Doc implementation generates pseudo-documents without any knowledge of the target corpus structure.

Future work should investigate injecting structural information from the knowledge base—such as passage hierarchy, article boundaries, section headings, and metadata—into the query enhancement prompt. This “chunking-aware” approach would enable the LLM to generate context-aware expansions tailored to the actual content organisation of the knowledge base, potentially bridging the gap between query-side and corpus-side optimisation.

2. **BM25 query repetition and retriever-specific strategies.** *Implemented in Phase 4.* The query repetition technique was investigated and fully resolved the BM25 term-dilution problem for all nine models (Section 4.6). Retriever-specific expansion strategies were further developed through the asymmetric CSQE application described in Section 4.9.
3. **Stronger embedding models.** The effectiveness of query enhancement should be evaluated with state-of-the-art Arabic-capable embedding models such as BGE-M3 and mE5-large, which are fine-tuned specifically for retrieval tasks. This evaluation would establish whether query enhancement provides additive improvement even over strong retrievers, or whether the benefits diminish as the baseline retriever becomes more capable.
4. **Hybrid retrieval with query enhancement.** *Implemented in Phase 4.* Hybrid BM25–Dense fusion was combined with corpus-steered query expansion, achieving 0.7137 NDCG@10—the best result in this thesis (Section 4.9).
5. **Dialectal Arabic evaluation.** The effectiveness of query enhancement for dialectal Arabic queries should be evaluated using datasets that include regional Arabic varieties. The vocabulary expansion mechanism of Query2Doc may help bridge MSA–dialect gaps by generating MSA-vocabulary pseudo-documents from dialectal queries, effectively serving as an implicit dialect-to-MSA normalisation layer. Datasets containing dialectal queries with MSA knowledge bases would be required to test this hypothesis.
6. **Few-shot and chain-of-thought prompting.** The current work used exclusively zero-shot prompting for pseudo-document generation. Few-shot prompting with curated Arabic demonstration examples may improve pseudo-document quality by

providing the model with explicit patterns for Arabic query expansion. Chain-of-thought reasoning before expansion—where the model analyses the query’s information needs before generating the pseudo-document—may further improve quality for complex or ambiguous queries.

7. **Multi-stage query enhancement.** The current approach applies a single round of query enhancement. Future work should investigate iterative refinement, where the results of a first retrieval pass inform a second round of query enhancement through a relevance feedback loop. This multi-stage approach could combine Query2Doc with re-ranking techniques, using the initially retrieved passages to generate more targeted query expansions.
8. **Publication and reproducibility.** The experimental finding that small open-source models can outperform the original Query2Doc paper’s results—achieved with a 175B-parameter proprietary model and few-shot prompting—for Arabic query enhancement represents a contribution suitable for publication. A pre-print submission or a faculty journal publication is recommended to disseminate these findings to the Arabic NLP research community before the rapidly evolving landscape of LLM-based retrieval renders the results less novel.
9. **First-pass quality gate.** A quality filter should be implemented that checks lexical overlap between the top-1 BM25 document and the query before grounding the LLM expansion. When overlap falls below a threshold, the system should fall back to blind QE. This is expected to recover approximately 131 Type B regression queries identified in the error analysis (Section 4.10.4).
10. **Asymmetric expansion weighting.** For Type A regressions (strong BM25 queries), reducing the expansion weight α or using field-based BM25 indexing (original query in one field, expansion in another with lower boost) would prevent vocabulary dilution for queries that BM25 already handles well (Section 4.10.4).
11. **CSQE with stronger dense retrievers.** The current pipeline uses mDPR as a relatively weak dense baseline (Section 3.2.1). Testing CSQE+Hybrid with BGE-M3 or mE5-large would assess whether the 54.5% BM25 improvement translates to similar gains over stronger dense retrievers, and whether the retriever-specific representation

finding generalises.

Bibliography

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” arXiv.org, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [2] M. Song and M. Zheng, “A survey of query optimization in large language models,” arXiv.org, 2024. [Online]. Available: <https://arxiv.org/abs/2412.17558>
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” arXiv.org, 04 2021. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [4] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [5] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” arXiv.org, 2020. [Online]. Available: <https://arxiv.org/abs/2004.04906>
- [6] L. Wang, N. Yang, and F. Wei, “Query2doc: Query expansion with large language models,” arXiv.org, 10 2023. [Online]. Available: <https://arxiv.org/abs/2303.07678>
- [7] S. Bruch, S. Gai, and A. Ingber, “An analysis of fusion functions for hybrid retrieval,” *ACM Transactions on Information Systems*, vol. 42, no. 1, 2024, arXiv:2210.11934.
- [8] L. Gao, X. Ma, J. Lin, and J. Callan, “Precise zero-shot dense retrieval without relevance labels,” 12 2022. [Online]. Available: <https://arxiv.org/pdf/2212.10496>

-
- [9] I. Mackie, S. Chatterjee, and J. Dalton, “Generative relevance feedback with large language models,” in *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2023.
 - [10] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, “Query rewriting for retrieval-augmented large language models,” arXiv.org, 10 2023. [Online]. Available: <https://arxiv.org/abs/2305.14283>
 - [11] S. R. El-Beltagy and M. A. Abdallah, “Exploring retrieval augmented generation in arabic,” arXiv.org, 2024. [Online]. Available: <https://arxiv.org/abs/2408.07425>
 - [12] J. Alsubhi, M. D. Alahmadi, A. Alhusayni, I. Aldailami, I. Hamdine, A. Shabana, Y. Iskandar, and S. Khayyat, “Optimizing rag pipelines for arabic: A systematic analysis of core components,” arXiv.org, 2025. [Online]. Available: <https://arxiv.org/abs/2506.06339v1>
 - [13] Technology Innovation Institute, “Falcon-h1: A family of hybrid-head language models,” arXiv.org, 2025. [Online]. Available: <https://arxiv.org/abs/2507.22448>
 - [14] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” arXiv.org, 2024. [Online]. Available: <https://arxiv.org/abs/2312.00752>
 - [15] N. Sengupta, S. K. Sahu, B. Jia, S. Katipomu, H. Li, F. Krupber, W. Mahboob, W. Zuo, N. Lubner, C. Axmed *et al.*, “Jais and jais-chat: Arabic-centric foundation and instruction-tuned open generative large language models,” arXiv.org, 2023. [Online]. Available: <https://arxiv.org/abs/2308.16149>
 - [16] M. S. Bari, Y. Alnumay, N. A. Alzahrani, N. M. Alotaibi, H. A. Alyahya, S. AlRashed, F. A. Mirza, S. Z. Alsubaie, H. A. Alahmed, G. Alabduljabbar *et al.*, “ALLam: Large language models for arabic and english,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=MscdsFVZrN>
 - [17] SILMA AI, “SILMA kashif-2b-instruct: Arabic RAG-optimized language model,” 2024, model card, accessed March 2026. [Online]. Available: <https://huggingface.co/silma-ai/SILMA-Kashif-2B-Instruct-v1.0>

-
- [18] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei *et al.*, “Qwen2.5 technical report,” arXiv.org, 2024. [Online]. Available: <https://arxiv.org/abs/2412.15115>
 - [19] Qwen Team, “Qwen3 technical report,” arXiv.org, 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
 - [20] Gemma Team, Google DeepMind, “Gemma 3 technical report,” arXiv.org, 2025. [Online]. Available: <https://arxiv.org/abs/2503.19786>
 - [21] J. Dang, V. Aryabumi, S. Dash, E. Beeching, S. Shandilya, A. Üstün, A. P. Singh, A. Bapna *et al.*, “Aya expand: Connecting billions of people through language technology,” arXiv.org, 2024. [Online]. Available: <https://arxiv.org/abs/2501.01482>
 - [22] OpenAI, “GPT-OSS,” arXiv.org, 2025. [Online]. Available: <https://arxiv.org/abs/2508.10925>
 - [23] X. H. Lü, “BM25S: An efficient python implementation of BM25,” 2024, software library. [Online]. Available: <https://github.com/xhluca/bm25s>
 - [24] I. Mackie, S. Chatterjee, and J. Dalton, “Generative and pseudo-relevant feedback for sparse, dense and learned sparse retrieval,” arXiv.org, 2023. [Online]. Available: <https://arxiv.org/abs/2305.07477>
 - [25] X. Lei, Z. Lian, X. Zhang, and Y. Lin, “Corpus-steered query expansion with large language models,” arXiv.org, 2024. [Online]. Available: <https://arxiv.org/abs/2402.18031>
 - [26] J. Zhang, M. Deng, Y. Zhou, S. Wang, and W. Li, “MUGI: Multi-granularity query expansion with llms for boosting sparse and dense retrieval,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024. [Online]. Available: <https://aclanthology.org/2024.findings-emnlp.103>
 - [27] Q. Zhang, F. Chen, Q. Yang, Y. Liu, and C. Wu, “Personalized query expansion with llms for dense and sparse retrieval,” arXiv.org, 2025. [Online]. Available: <https://arxiv.org/abs/2510.08935>

-
- [28] S. Yoon, S. Kim, and J. Park, “On the role of llms in query expansion for information retrieval,” arXiv.org, 2025. [Online]. Available: <https://arxiv.org/abs/2504.14175>
 - [29] Y. Xia, J. Wang, Y. Zhang, and D. Li, “Knowledge-augmented retrieval: Bridging the gap between query and document representations,” 2025. [Online]. Available: <https://aclanthology.org/2025.naacl-long.216>
 - [30] Z. Yang, X. Zhang, and Y. Huang, “Aligned query expansion for dense retrieval,” arXiv.org, 2025. [Online]. Available: <https://arxiv.org/abs/2507.11042>
 - [31] X. Lei, Z. Lian, and X. Zhang, “ThinkQE: Chain-of-thought query expansion with large language models,” 2025. [Online]. Available: <https://aclanthology.org/2025.findings-emnlp.965>
 - [32] O. Young, Y. Fan, R. Zhang, J. Guo, M. de Rijke, and X. Cheng, “GaQR: An efficient generation-augmented question rewriter,” in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM '24)*, 2024, pp. 4228–4232.
 - [33] C.-M. Chan, C. Xu, R. Yuan, H. Luo, W. Xue, Y. Guo, and J. Fu, “Rq-rag: Learning to refine queries for retrieval augmented generation,” arXiv.org, 03 2024. [Online]. Available: <https://arxiv.org/abs/2404.00610>
 - [34] L. Louis and A. Bhusal, “Query augmentation for information retrieval (ir) using large language model (llm),” 2024.
 - [35] K. Zhang, Z. Sun, W. Yu, X. Zang, K. Zheng, Y. Song, H. Li, and J. Xu, “Qe-rag: A robust retrieval-augmented generation benchmark for query entry errors,” arXiv.org, 2025. [Online]. Available: <https://arxiv.org/abs/2504.04062>
 - [36] X. Zhang, N. Thakur, O. Ogundepo, E. Kamalloo, D. Alfonso-Hermelo, X. Li, Q. Liu, M. Rezagholizadeh, and J. Lin, “Miracl: A multilingual retrieval dataset covering 18 diverse languages,” arXiv.org, 2023. [Online]. Available: <https://arxiv.org/abs/2210.09984>
 - [37] J. Lin, X. Ma, S.-C. Lin, J.-H. Yang, R. Pradeep, and R. Nogueira, “Pyserini: A python toolkit for reproducible information retrieval research

- with sparse and dense representations,” arXiv.org, 2021. [Online]. Available: <https://arxiv.org/abs/2102.10073>
- [38] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized language models,” arXiv.org, 2023. [Online]. Available: <https://arxiv.org/abs/2305.14314>